Basi di dati

(aiuto!!!)

Matteo Balzerani

October 2025

Contents

I Teoria **2**

1 Introduzione **2**
1.1 Moduli di un DBMS 3
1.2 Condivisione di una BD 3
1.3 Gestione della memoria secondaria 4
1.3.1 Buffer 4
1.3.2 Gestione del buffer 4

2 Gestione delle TUPLE nelle pagine **6**
2.1 Strutture sequenziali 6
2.1.1 Struttura seriale (disordinata) 6
2.1.2 Struttura ad array 7
2.1.3 Strutture ordinate (clustered) 7
2.2 Strutture ad accesso calcolato: HASH . . . 7
2.2.1 File HASH 8
2.3 Strutture ad Albero 9
2.3.1 Indici di file 9
2.4 Strutture ad albero dinamiche 11
2.4.1 Albero binario di ricerca 11
2.4.2 Albero di ricerca di ordine Q 11
2.4.3 B-Tree 12
2.4.4 B+ Tree 18

3 Gestione delle transazioni **22**
3.1 Proprietà ACID delle transazioni 22
3.1.1 Atomicità 22
3.1.2 Consistenza 23
3.1.3 Isolamento 23
3.1.4 Durabilità (persistenza) 23
3.2 Gestore delle transazioni 23
3.2.1 Gestore dell'affidabilità 24
3.2.2 Il log 24
3.2.3 UNDO e REDO 25
3.2.4 Checkpoint 25
3.2.5 Dump 26
3.2.6 Esito di una transazione 26
3.2.7 Regole fondamentali per il log 26

4 Gestione dei guasti **27**
4.1 Ripresa a caldo 28
4.2 Ripresa a freddo 29

5 Gestore della concorrenza **29**
5.1 Anomalie delle transazioni concorrenti . . . 30
5.1.1 Perdita di aggiornamento 30
5.1.2 Lettura sporca 31
5.1.3 Letture inconsistenti 31
5.1.4 Aggiornamento fantasma 31
5.2 Controllo di Concorrenza 32
5.2.1 Soluzioni iniziali NON applicabili . . 32
5.2.2 Soluzioni utilizzate in pratica: LOCKING 35
5.2.3 Meccanismi di locking 35
5.2.4 Locking a due fasi stretto 36
5.2.5 Stallo (deadlock) - blocco critico . . 36

6 Possibili domande d'esame **37**

Descrizione Esame

- **Modulo di Teoria** Prova scritta che consiste in domande ed esercizi sugli argomenti trattati. Possibilità di ripetere la prova. In caso di ripetizione della prova, l'ultimo voto ottenuto sostituisce il precedente.
- **Modulo di Laboratorio** Prova scritta che consiste in esercizi sugli argomenti trattati. Il voto finale dell'insegnamento di Basi di Dati e Web sarà dato dalla media tra il voto della prova scritta di teoria e il voto delle prova scritta di laboratorio. Il voto complessivo verrà pubblicato anche sulla piattaforma MyUnivr e dopo l'accettazione verrà verbalizzato.

Part I
Teoria

1 Introduzione

Cos'è un DBMS? Sistema (prodotto software) in grado di gestire collezioni di dati che siano:

- **grandi** (di dimensioni (molto) maggiori della memoria centrale dei sistemi di calcolo utilizzati). La grandezza richiede che tale gestione sia sofisticata (non possiamo caricare tutto in memoria principale e poi riscaricare).
- **persistenti** (con un periodo di vita indipendente dalle singole esecuzioni dei programmi che le utilizzano). La persistenza richiede una gestione in memoria secondaria.
- **condivise** (utilizzate da applicazioni diverse)

garantendo **affidabilità** (resistenza a malfunzionamenti hardware e software) e **privatezza** (con una disciplina e un controllo degli accessi). Come ogni prodotto informatico, un DBMS deve essere **efficiente** (utilizzando al meglio le risorse di spazio e tempo del sistema) ed **efficace** (rendendo produttive le attività dei suoi utilizzatori).

Architettura a livelli dei DBMS I DBMS sono organizzati secondo una architettura a livelli

- Livello **LOGICO**: a cui fanno riferimento gli utenti (SQL).
- Livello **FISICO**: stabilisce l'effettiva implementazione in modo efficiente

  Esempio: le relazioni del modello relazionale dei dati vengono realizzate utilizzando varie possibili strutture fisiche

La proprietà di INDIPENDENZA dei dati permette agli utenti di ignorare le strutture fisiche – SQL fa riferimento al livello logico – Il DBMS si preoccupa di tradurre le operazioni al livello più basso, utilizzando in modo efficiente le strutture fisiche. Gli utenti vedono il modello logico (relazionale).
I dati sono in **memoria secondaria**, le strutture logiche non sarebbe efficienti in memoria secondaria: servono strutture fisiche opportune. La memoria secondaria è molto più lenta della memoria principale: serve un'interazione fra memoria principale e secondaria che limiti il più possibile gli accessi alla secondaria.

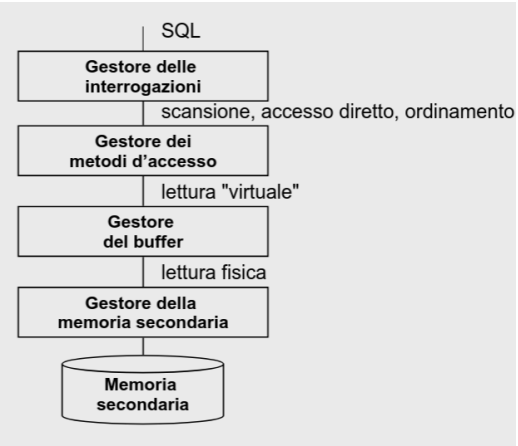


Figure 1: Architettura della gestione degli accessi e delle interrogazioni in un DBMS

1.1 Moduli di un DBMS

Domanda d'esame: ***Si descrivano i moduli che compongono il DBSM descrivendo brevemente le funzioni di ciascuno:*** risposta nelllo schema sottostante. Bisogna parlare di ciascun modulo brevemente, facendo l'esempio di una query dall'alto verso il basso e viceversa.

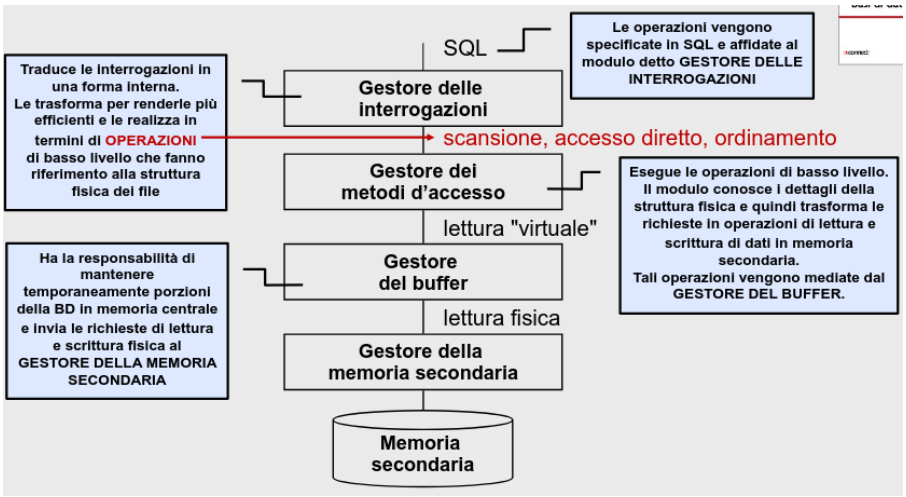


Figure 2

Gestore delle interrogazioni Prende la query logica e la traduce in una forma interna. Le trasforma per renderle più efficienti e le realizza in termini di **operazioni di basso livello** che fanno riferimento alla struttura fisica dei file.

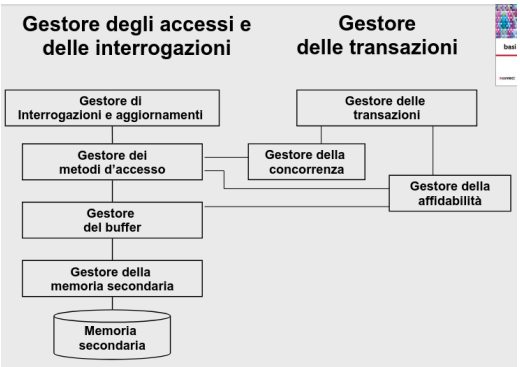
Gestore dei metodi di accesso Esegue le operazioni di basso livello. Il modulo conosce i dettagli della struttura fisica e quindi trasforma le richieste in operazioni di lettura e scrittura di dati in memoria secondaria. Tali operazioni vengono mediate dal GESTORE DEL BUFFER.

Gestore del buffer Ha la responsabilità di manetenere temporaneamente porzioni della BD in memoria centrale e invia le richieste di lettura e scrittura fisica al **gestore della memoria secondaria.**

Transazioni le basi di dati si basano sul concetto di transazione (insieme di operazioni), che deve essere **atomiche** (o tutto o niente) e **definitive** (dopo che è andata a buon fine, deve rimanere permanente la transazione eseguita).

1.2 Condivisione di una BD

Una BD è una risorsa *integrata*, *condivisa* fra le varie applicazioni, e di conseguenza vi sono attività diverse su dati in parte condivisi; questo richiede **meccanismi di autorizzazione** e attività multi-utente su dati condivisi: richiedono **controllo della concorrenza.** L'accesso concorrente in scrittura deve essere regolato.



Il **gestore della concorrenza e il gestore della affidabilità sono sottomoduli del gestore delle transazioni** (DBMS sistema informativo transazionale.) Significa gestire la concorrenza e gestire l'affidabilità.

Domanda: *Quali sono i moduli che mi permettono di gestire le transazioni in un DBMS?*

1.3 Gestione della memoria secondaria

Il costo di un accesso a memoria secondaria è quattro o più ordini di grandezza maggiore di quello per operazioni in memoria centrale. Nelle applicazioni "I/O bound" (cioè con molti accessi a memoria secondaria e relativamente poche operazioni) il costo dipende esclusivamente dal numero di accessi a memoria secondaria. Nei DBMS l'interazione tra memoria centrale e secondaria è realizzata attraverso un buffer.

1.3.1 Buffer

- Area di memoria centrale, gestita dal DBMS (preallocata) e condivisa fra le transazioni.
- Organizzato in **pagine** di dimensioni pari o multiple di quelle dei blocchi di memoria secondaria. (Per semplicità ipotizzeremo che ciascuna pagina corrisponda ad un blocco)
- L'allocazione di buffer sempre più grandi, riduce il numero di accessi alla memoria secondaria
- La gestione ottimale del buffer è un aspetto fondamentale nelle BD (rispetto alle prestazioni). Il buffer è importantissimo per via della grande differenza di tempo di accesso fra memoria centrale e memoria secondaria.

1.3.2 Gestione del buffer

Scopo della gestione del buffer

- **Ridurre il numero di accessi alla memoria secondaria** *GARANTENDO L'AFFIDABILITÀ.*
  - In caso di **lettura**, se la pagina è già presente nel buffer, non è necessario accedere alla memoria secondaria.
  - In caso di **scrittura**, il gestore del buffer può decidere di differire la scrittura fisica (ammesso che ciò sia compatibile con la gestione dell'affidabilità)
- Le politiche di gestione del buffer assomigliano a quelle della gestione della memoria centrale da parte dei Sistemi Operativi: obbediscono allo stesso principio, detto di località, in base al quale i dati referenziati di recente hanno una maggiore probabilità di essere referenziati nuovamente in futuro

Il buffer offre ai moduli di livello più alto una interfaccia complessa, non si limita alle richieste di lettura e scrittura. Ha bisogno delle informazioni sul **prevedibile riutilizzo delle pagine** (per evitarne la riapertura); Ha bisogno delle informazioni sulla necessità di **effettuare immediatamente gli aggiornamenti** (per evitare la perdita in caso di guasto). Per la gestione dei buffer sono necessarie le seguenti informazioni:

- Un direttorio che per ogni pagina mantiene (ad esempio) il file fisico e il numero del blocco ad essa corrispondente.
- Per ogni pagina del buffer il gestore mantiene due variabili di stato: **un contatore** che indica quanti programmi utilizzano la pagina e un **bit** che indica se la pagina è "sporca", cioè se è stata modificata (*dirty bit*).

Funzioni del buffer manager Le politiche sono simili a quelle relative alla gestione della memoria da parte dei sistemi operativi.

Principio di "località dei dati": è alta la probabilità di dover riutilizzare i dati attualmente in uso.
Legge 80-20: l'80% delle operazioni utilizza sempre lo stesso 20% dei dati.

Operazioni per la gestione del buffer:

- **Fix:** Operazione che permette a un processo di accedere a una pagina, caricandola dal disco solo se non è già presente nel buffer. Incrementa il contatore dei riferimenti (pin count) della pagina per bloccarla ed evitare che venga rimossa mentre è in uso. Restituisce il riferimento (indirizzo) alla pagina nel buffer.

  Passi dell'operazione:

  1. **Cerca la pagina nel buffer.**
     - Se è presente (grazie al principio di località, ciò avviene abbastanza spesso): incrementa il contatore e restituisce l'indirizzo.
     - Se non è presente: procedi al passo successivo.
  2. **Cerca un frame libero** (cioè una pagina con contatore = 0, non usata da nessun processo).
     - Se lo trova:
       * Se il dirty bit (bit di stato modificato) è 1, riscrive la pagina in memoria di massa prima di sovrascriverla.
       * Carica la nuova pagina dal disco, imposta il contatore a 1, restituisce l'indirizzo.
     - Se non trova frame liberi (tutti i contatori sono > 0): adotta una delle due strategie:
       * **Steal:** seleziona una pagina "vittima" (es. con algoritmo LRU), la scrive su disco se modificata, la sovrascrive con la nuova pagina, restituisce l'indirizzo.
       * **No-steal:** mette l'operazione in attesa finché non si libera un frame (cioè qualche processo esegue Unfix portando un contatore a 0).

- **setDirty** Comunica al buffer manager che la pagina è stata modificata. Viene modificato il BIT di stato relativo.

- **Unfix:** indica che la transazione ha concluso l'utilizzo della pagina, decrementa il contatore associato alla pagina.

- **Force:** Trasferisce in modo sincrono una pagina in memoria secondaria **su richiesta del gestore dell'affidabilità** (non del gestore degli accessi) per garantire che alcuni dati non vengano persi. Memorizza in maniera permanente le modifiche per garantire l'affidabilità.

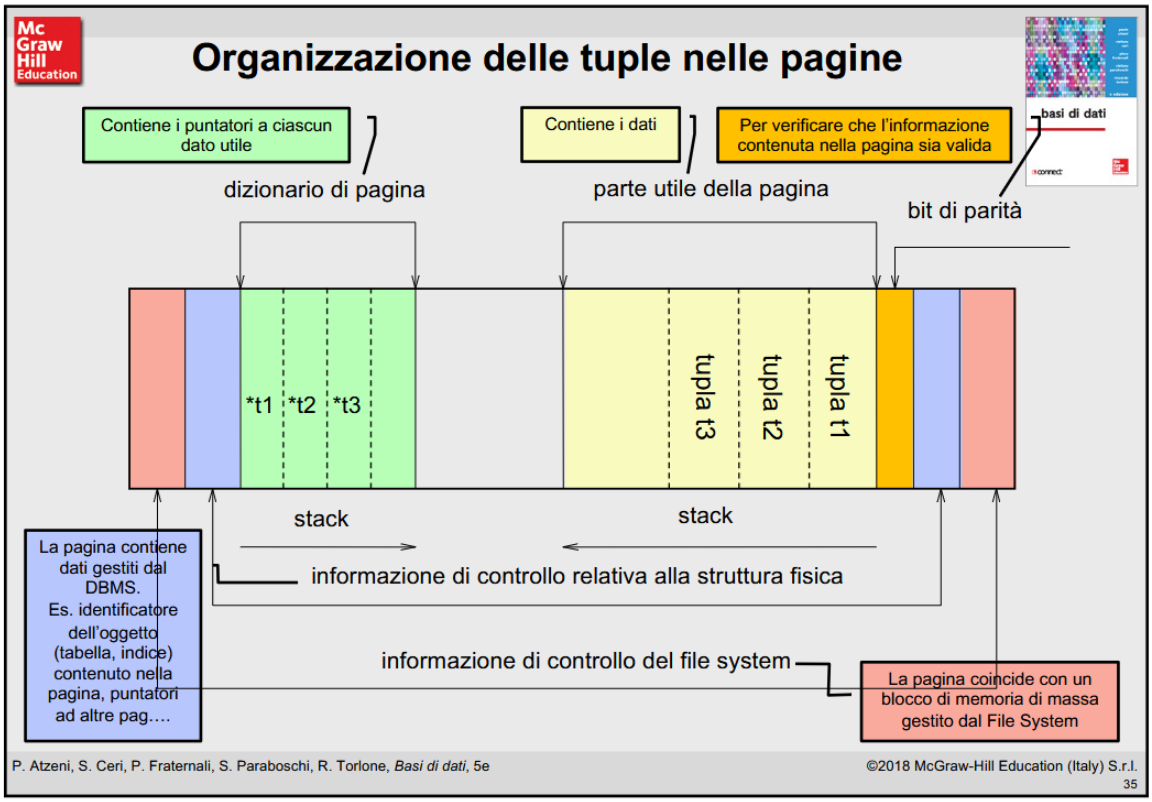
La scrittura in memoria secondaria può avvenire in due modi:

- in modo **sincrono** quando è richiesto esplicitamente con una force
- in modo **asincrono** (indipendente dall'applicazione) quando il buffer manager lo ritiene opportuno (o necessario). In particolare, può decidere di anticipare o posticipare scritture per coordinarle e/o sfruttare la disponibilità dei dispositivi

2 Gestione delle TUPLE nelle pagine

Ciascuna pagina contiene l'*informazione utile*, cioè i dati veri e proprio, e l'*informazione di controllo*, per accedere all'informazione utile. I blocchi (componenti fisici di un file) e i record (componenti logici) hanno dimensioni in generale diverse: la dimensione del blocco *dipende dal file system* e la dimensione del record dipende dalle esigenze del record.

Fattore di blocco Il numero di **record** in un **blocco** è pari al rapporto fra la dimensione del blocco L_B e la dimensione media dei record L_R. Se $L_B > L_R$ possiamo avere più record in un blocco:$\lfloor L_B/L_R \rfloor$. Lo spazio residuo può essere *utilizzato*, cioè **spanned** (impaccato) o non utilizzato, cioè **unspanned**



Abbiamo diverse strutture primarie per l'organizzazione di file, e stabiliscono il *criterio secondo cui le tuple sono disposte nel file*. Ci sono tre categorie principali di strutture:

- **Sequenziali:** disposizione consecuitva delle tuple
- **Ad accesso calcolato - HASH:** disposizione in posizioni che dipendono dall'esecuzione di un algoritmo
- **Ad albero:** usate soprattutto come **strutture secondarie.**

2.1 Strutture sequenziali

Nelle strutture sequenziali un file è costituito da vari blocchi di memoria "logicamente" consecutivi. Le tuple vengono inserite nei blocchi rispettando una sequenza.

2.1.1 Struttura seriale (disordinata)

È la struttura sequenziale più semplice ma anche più diffusa, le tuple venogno inserite nel file nell'ordine in cui si presentano; non esiste un ordinamento basato su qualche valore. Viene anche chiamata **"entry sequenced"**/file heap/file disordinato. Gli inserimento vengono effettuati *in coda* o *al posto di record cancellati*. Le **cancellazioni** si effettuano marcando il record come "cancellato". le tuple vengono periodicamente riorganizzate per eliminare lo spazio sprecato.
È molto diffusa nelle basi di dati relazionali, associata ad inidici secondari. È molto efficiente per le operazioni di inserimento, poiché è sufficiente mantenere un riferimento all'ultimo blocco. Le *ricerche sono poco efficienti:* richiedono una scansione sequenziale di tutti i record, con un costo lineare nel numero di blocchi del file. Per questo motivo sono spesso usate insieme a strutture secondarie usate per favorire gli accessi.

2.1.2 Struttura ad array

È possibile solo quando le tuple hanno lunghezza fissa: al file viene associato un numero di blocchi contigui; ogni *blocco* ha un numero fisso di posizioni disponibili per le tuple; ogni *tupla* ha un indice i e viene posta nella posizione dell'indice associato. Questa struttura non è quasi mai utilizzata dai DBMS anche se sono efficienti per la ricerca.

2.1.3 Strutture ordinate (clustered)

Prevedono la memorizzazione dei record secondo un **ordinamento fisico coerente con l'ordinamento di un campo detto CHIAVE.** Permettono ricerche binarie, ma solo fino ad un certo punto, poiché senza indici non conosci la disposizione dei blocchi, quindi non puoi *calcolare la posizione a metà di un file.* Per questo, nelle basi di dati relazionali si utilizzano quasi solo in combinazioni con indici come **file ISAM** o file ordinati con *indice primario.* ISAM è l'acronimo di *Indexed Sequential Access Method.*

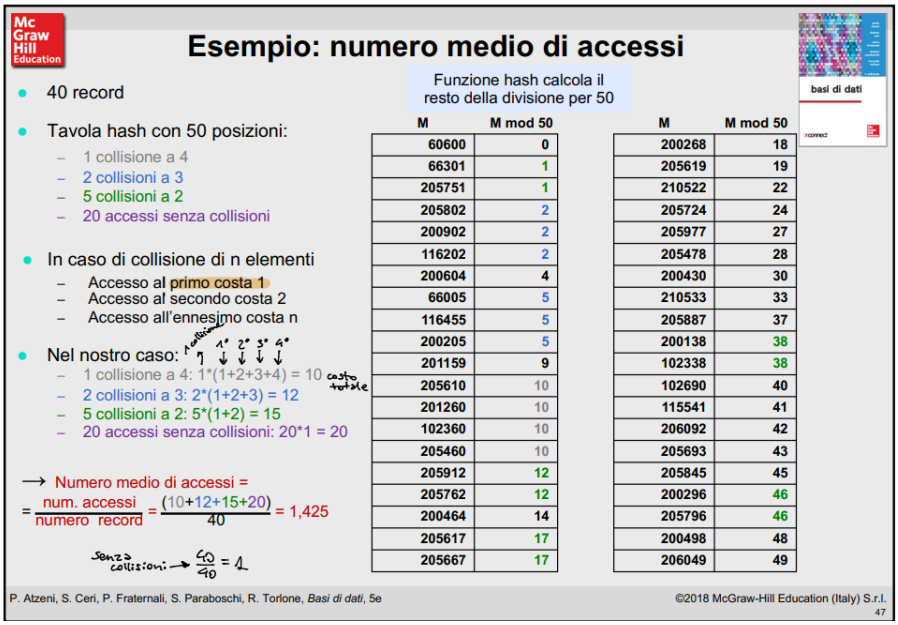
2.2 Strutture ad accesso calcolato: HASH

Le tuple sono collocate in posizioni che dipendono dall'esecuziuone di un algoritmo. Permettono un accesso diretto molto efficiente. La tecnica si basa su quella utilizzata per le tavole hash in memoria centrale.
Obiettivo: accesso diretto ad un insieme di record sulla base del valore di un campo (detto chiave, che per semplicità supponiamo identificante, ma non è necessario).

- Se i possibili valori della chiave sono in numero paragonabile al numero di record (e corrispondono ad un "tipo indice") allora usiamo un array. *Esempio: università con 1000 studenti e numeri di matricola compresi fra 1 e 1000 o poco più e file con tutti gli studenti.*
- Se i possibili valori della chiave sono molti di più di quelli effettivamente utilizzati, non possiamo usare l'array (spreco); *ad esempio: 40 studenti e numero di matricola di 6 cifre (un milione di possibili chiavi).*

Volendo continuare ad usare qualcosa di simile ad un array,ma senza sprecare spazio, possiamo pensare di trasformare i valori della chiave in possibili indici di un array: **Funzione HASH**.

Funzione HASH Associa ad ogni valore della chiave un "indirizzo", in uno spazio di dimensione paragonabile (leggermente superiore) rispetto a quello strettamente necessario. La funzione non può essere iniettiva, quindi il numero di possibili chiavi è molto maggiore del numero di possibili indirizzi ("lo spazio delle chiavi è più grande dello spazio degli indirizzi"), ed esiste la possibilità di collisioni (chiavi diverse che corrispondono allo stesso indirizzo). Le buone funzioni hash distribuiscono in modo causale e uniforme, riducendo le probabilità di collisione (che si riduce aumentando lo spazio ridondante). Varie tecniche: posizioni successive disponibili, tabella di overflow (gestita in forma collegata), funzioni hash "alternative".



Nota:

- le collisioni ci sono (quasi) sempre
- le collisioni multiple hanno probabilità che decresce al crescere della molteplicità
- la molteplicità media delle collisioni è molto bassa

2.2.1 File HASH

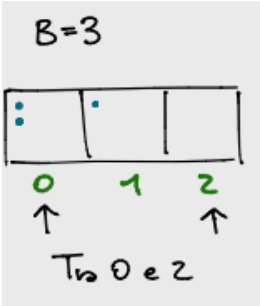
L'idea è la stessa della tavola hash, ma si basa sull'organizzazione in blocchi. L'operazione di costo unitario è l'accesso ad un blocco che può contenere diversi record. I blocchi sono riempiti solo in parte:

- **Fattore di riempimento FdR:** frazione dello spazio fisico disponibile mediamente utilizzato. *Ad esempio: 40 record di studenti su 50 posizioni disponibili* $\rightarrow FdR = 40/50 = 0.80$
- **Fattore di blocco FdB:** rapporto fra la dimensione del blocco L_B e la dimensione del record L_R: $FdB = \lfloor L_B/L_R \rfloor$

Se supponiamo di avere:

- **T:** numero di tuple previsto per il file
- **FdB:** fattore di blocco
- **FdR:** fattore di riempimento

Allora il file può avere un numero di **Blocchi B** pari a $B = \lceil \frac{T}{FdR \cdot FdB} \rceil$
La funzione di HASH ritorna un numero compreso tra 0 e $B - 1$, interpretato come il numero del blocco nell'ambito dei blocchi allocati al file.



I **blocchi allocati al file** sono **B**, la funzione di HASH ritorna un numero tra **0 e B-1**. In caso di collisione i record vengono semplicemente allocati nel blocco fino ad esaurimento dello spazio libero; vengono accettate FdB (numero di record nel blocco) collisioni. Quando un blocco viene esaurito viene allocato un nuovo blocco collegato al precedente ed il record viene disposto nel nuovo blocco: *CATENE DI OVERFLOW*.

È **l'organizzazione più efficiente per l'accesso diretto basato su valori della chiave** con condizioni di uguaglianza (accesso puntuale): il costo medio è di poco superiore all'unità e il caso peggiore è molto costoso ma talmente improbabile da poter essere ignorato. *Le collisioni (overflow) sono di solito gestite con blocchi collegati.* **Non è efficiente per ricerche basate su intervalli**: le funzioni HASH tendono a collocare valori vicini su blocchi diversi, ad esempio: ricerca degli studenti con matricola compresa tra 17010 e 17029 richiede 20 accessi diretti uno per ciascun valore dell'intervallo.

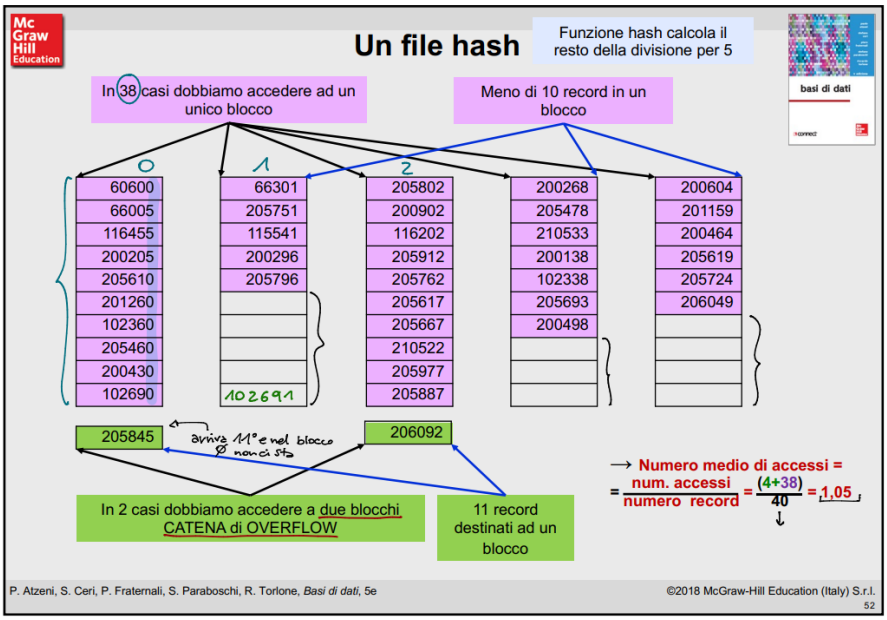


Figure 3: Esempio File hash. Numero tuple T = 40. Numero Blocchi B = 5.FdB = 10 (10 posizioni in ogni blocco). Posizioni disponibili = 50. FdR = 0.80. La funzione di HASH restituisce un numero tra 0 e 4. Numero di Overflow 2 → num. medio di accessi: 1,05

2.3 Strutture ad Albero

L'organizzazione ad albero può essere utilizzata per realizzare *strutture primarie (contenenti i dati)* e *strutture secondarie*, che favoriscono l'accesso ai dati senza contenere i dati stessi.

> È leggermente *meno efficiente* per accessi puntuali, ma mantiene la stessa efficienza anche per accessi basati su interalli.

2.3.1 Indici di file

Indice: struttura ausiliaria per l'*accesso efficiente* ai record di un file sulla base dei valori di un campo (o di una concatenazione di campi) detto **chiave** (o, meglio, pseudochiave, perché non è necessariamente identificante).

> **Idea fondamentale:** l'indice analitico di un libro, lista di coppie (termine, pagina), ordinata alfabeticamente sui termini, posta in fondo al libro e separabile da esso.

Un indice I di un file f è un **altro file**, con **record a due campi:** *chiave* e *indirizzo* (dei record di f o dei relativi blocchi), *ordinato* secondo i valori della chiave.

Tipi di indice A seconda della popolazione dell'indice abbiamo:

- **Indice denso:** contiene un record per ciascun valore del campo chiave.
- **Indice sparso:** contiene un numero di record *inferiore* rispetto al numero di valori diversi dal campo chiave, cioè *esistono valori della chiave che NON sono presenti nell'indice.*

Vi è poi la differenza tra:

- **Indice primario:** l'indice contiene al suo interno i dati o è realizzato su un file ordinato sullo stesso capmo su cui è definito l'indice stesso. *Può essere sparso.*

  Esempio: indice generale di un libro, determina la posizione dei capitoli che sono i dati del libro.

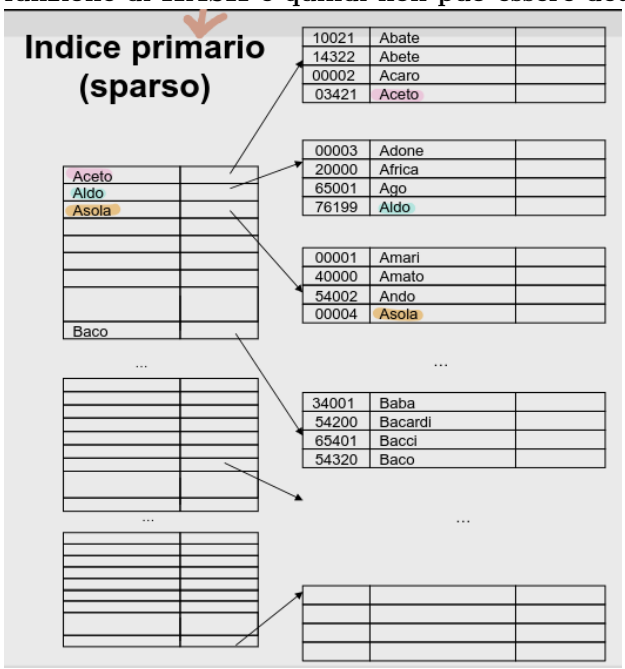
  Un indice primario, grazie all'ordinamento, può essere realizzato "puntando" ad un solo record di un blocco (indice sparso), poiché gli altri record sono posti in modo adiacente a quello puntato.

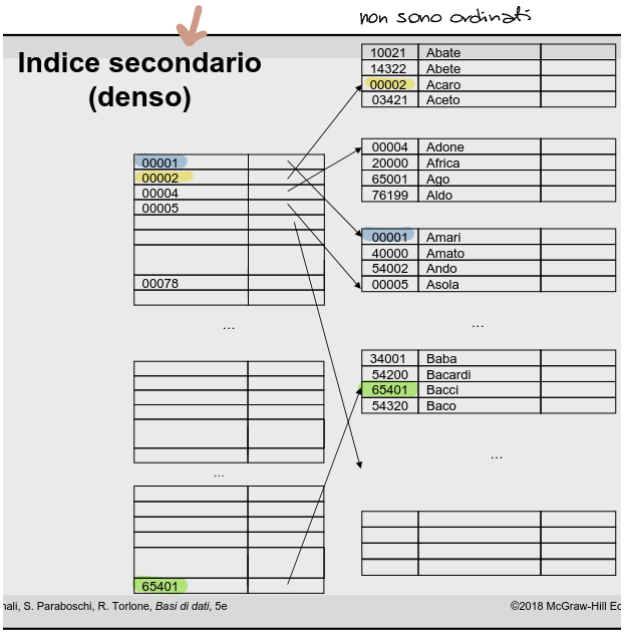
- **Indice secondario:** l'indice è definito su un campo con ordinamento diverso da quello di memorizzazione. **DEVE essere denso.**

  Esempio: indice analitico di un libro, che è l'indice secondario che garantisce l'accesso senza contenere i dati.

  Un indice secondario deve contenere tutti i rifermenti a tutti i valori della chiave (indice denso), infatti valori consecutivi della chiave possono trovarsi in blocchi diversi.

I benefici legati alla presenza di indici secondari sono molto più sensibili. *Ogni file può avere al più un indice primario e un numero qualunque di indici secondari (su campi diversi)..*
Un file hash non può avere un indice primario: Il criterio di memorizzazione è stabilito dalla funzione di HASH e quindi non può essere determinato dall'indice.





Domanda d'esame: si dia la definizione di indice, che cos'è un indice primario, secondario, denso e sparso.
*Risposta:*l'indice è un file che ha un insieme di coppie con valore di chiave e indirizzo, se l'indice contiene le informazioni è un indice primario, se non contiene le informazioni è secondario. Se contiene un Record per ogni valore della chiave è denso, se no è sparso. Gli indici primari possono essere sparsi. Gli indici secondari devono essere densi.

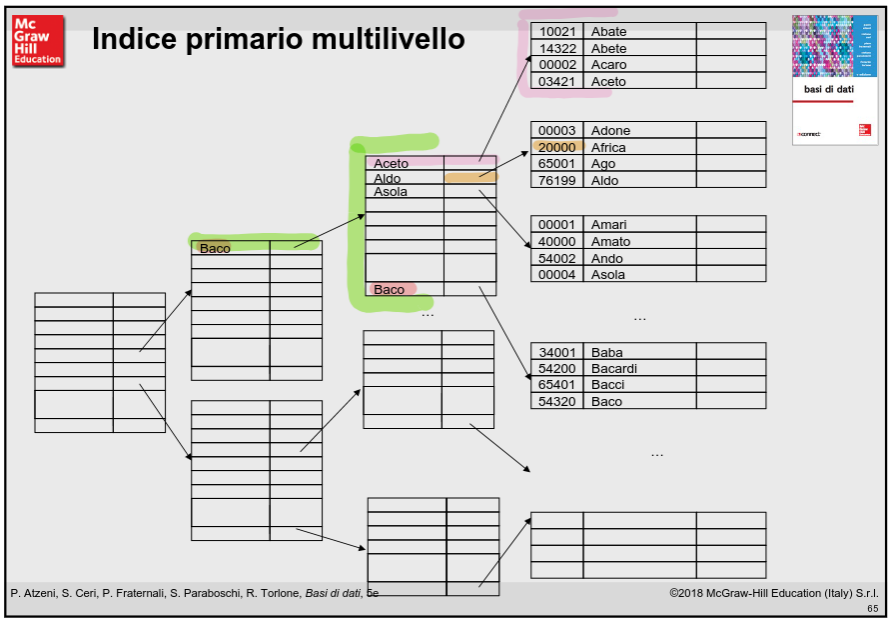
Prestazioni degli indici

- Accesso diretto sulla chiave efficiente, sia puntuale sia per intervalli
- scansione sequenziale ordinata efficiente: tutti gli indici (in particolare quelli secondari) forniscono un **ordinamento logico** sui record dei file.
- Modifiche della chiave, inserimenti, eliminazioni *inefficienti*: esistono tecniche per alleviare i problemi: *file o blocchi di overflow, marcatura per le eliminiazioni, riempimento parziale, blocchi collegati (non contigui), riorganizzazioni periodiche...*

Osservazione sugli indici secondari: si possono usare, come detto, puntatori ai blocchi oppure puntatori ai record:

- I puntatori ai blocchi sono più compatti
- I puntatori ai record permettono di semplificare alcune operazioni (effettuate solo sull'indice, senza accedere al file se non quando indispensabile)

Indici multilivello Gli indici sono file essi stessi e quindi ha senso costruire indici sugli indici, per evitare di fare ricerche fra blocchi diversi. Possono esistere più livelli fino ad avere il livello più alto con un solo blocco. I livelli sono di solito abbastanza pochi, perché l'indice è ordinato, quindi l'indice sull'indice è sparso, e i record dell'indice sono piccoli.



Tutte le strutture di indice viste finora sono basate su strutture ordinate e quindi sono poco flessibili in presenza di elevata dinamicità. Gli indici utilizzati dai DBMS sono più sofisticati. Si usano strutture al albero dinamiche. Sono efficienti in presenza di aggiornamenti.

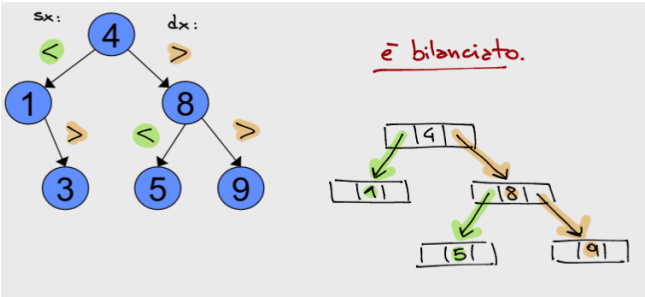
2.4 Strutture ad albero dinamiche

Ogni albero è caratterizzato da un **nodo radice**, vari **nodi intermedi**, e vari **nodi foglia**. Ogni nodo coincide con una *pagina o blocco* a livello di file system e di gestore del buffer. Ogni nodo ha un numero di figli abbastanza grande che dipende dall'ampiezza della pagina. I legami tra i nodi vengono stabiliti da *puntatori* che collegano fra loro le pagine.

> Requisito importante per il buon funzionamento di queste strutture è che devono essere **bilanciate**, ovvero che la *lunghezza del cammino che collega la radice alle foglie è* ***costante***, ovvero che il tempo di accesso è uguale per tutte le informazioni.

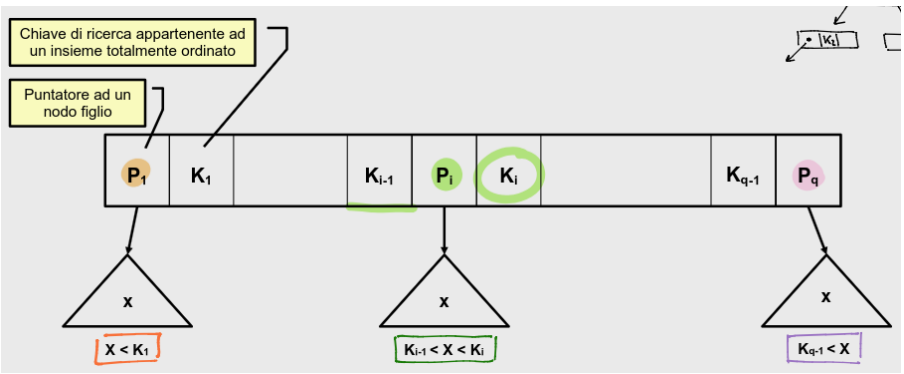
2.4.1 Albero binario di ricerca

È un albero binario etichettato in cui per ogni nodo il *sottoalbero sinistro* contiene solo **etichette** *minori di quella del nodo* e il *sotttoalbero destro* etichette maggiori. **Tempo di ricerca ed inserimento pari alla profondità.** Logaritmico nel caso "medio".



2.4.2 Albero di ricerca di ordine Q

Ogni nodo ha fino a **Q figli** e fino a **Q-1** etichette, ordinate.
Nell'i-esimo sottoalbero abbiamo tutte etichette magiori della (i-1)-esima etichetta e minori della i-esima.



2.4.3 B-Tree

Nei B-Tree i *nodi intermedi possono avere puntatori direttamente ai dati.* Ogni nodo ha la struttura seguente:

$$[p_0(k_1, r_1)p_1 ... p_{j-1}(k_j, r_j)p_j]$$

dove r_i è un indirizzo al record di chiave k_i e p_i è un puntatore ad un figlio.

- Le chiavi k_i sono in ogni nodo sono ordinate.
- Ogni nodo non foglia con j chiavi ha *j+1* figli
- Tutte le chiavi associate ad un figlio tramite il puntatore p_i sono comprese **strettamente** tra k_i e k_{i+1}.
- Ogni chiave fa riferimento ad un indirizzo, quindi nell'albero non ci sono chiavi duplicate.

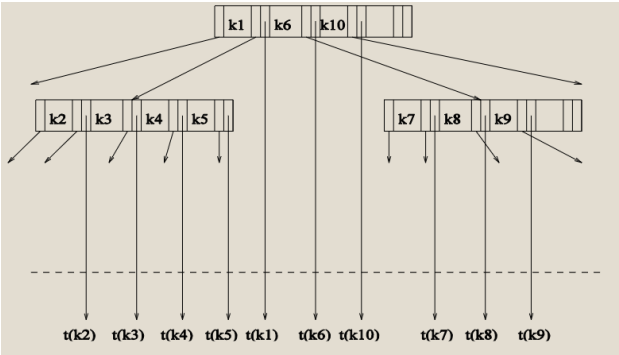


Figure 4: B-Tree con riferimento a valori intermedi

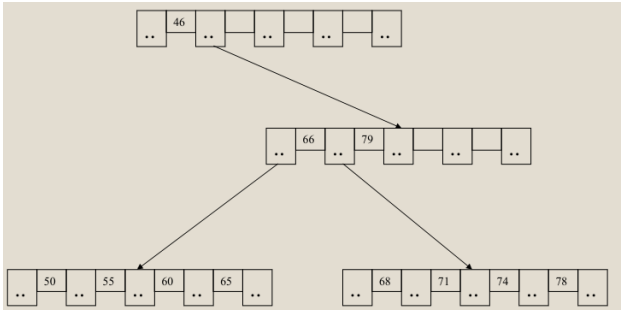


Figure 5: Estratto di B-tree di ordine 5

Definizione di B-Tree di ordine m (con m ≥ 3) - *ORDINE = FAN-OUT*

- La radice ha

$$0 \leq \textit{nodi figli} \leq m$$
$$1 \leq \textit{numero chiavi} \leq m - 1$$

- Ogni nodo interno ha

$$\lceil m/2 \rceil \leq \textit{numero di figli} \leq m$$
$$\lceil m/2 \rceil - 1 \leq \textit{numero di chiavi} \leq m - 1$$

- Tutte le foglie si trovano allo stesso livello

Inserimento in B-tree Si verifica se la chiave è presente, altrimenti si scende alla foglia appropriata. Se il nodo non è completo (cioè contiene meno di m-1 chiavi), allora vi si può inserire il record. Se il nodo è completo, si crea un altro nodo (splitting) e si considera il valore mediano tra le chiavi del nodo completo più la chiave da inserire:

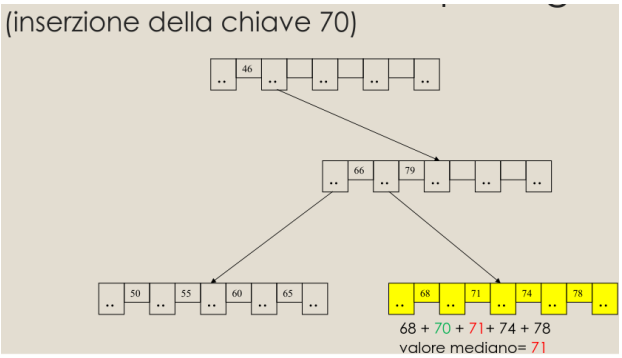
1. Il valore mediano si inserisce nel nodo padre
2. Se il padre è completo si esegue la suddivisione (splitting) del padre (processo ricorsivo)

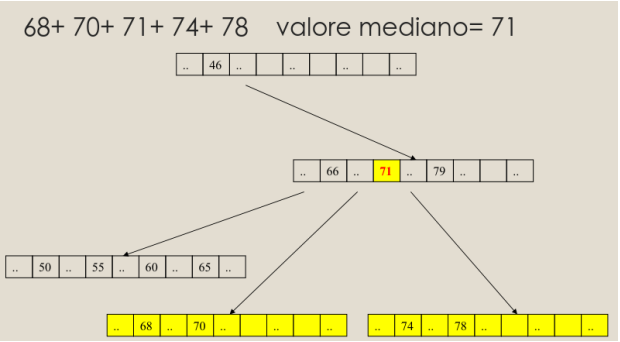
Inserimento in nodo saturo Se il nodo p' è saturo, cioè contiene m-1 chiavi, si esegue lo **splitting** del nodo:

- Si crea un altro nodo p" in cui si memorizzano le chiavi maggiori della chiave di *valore mediano* (oppure di posto mediano) k_c nella sequenza (considerando chiavi del nodo e chiave da inserire) togliendole dal nodo originario
- La chiave di valore mediano k_c si inserisce nel nodo padre, o meglio si inserisce

$$...p'(k_c, r_{kc})p''...$$

  e se il padre è completo si esegue la suddivisione (splitting) del padre e così via fino alla radice.

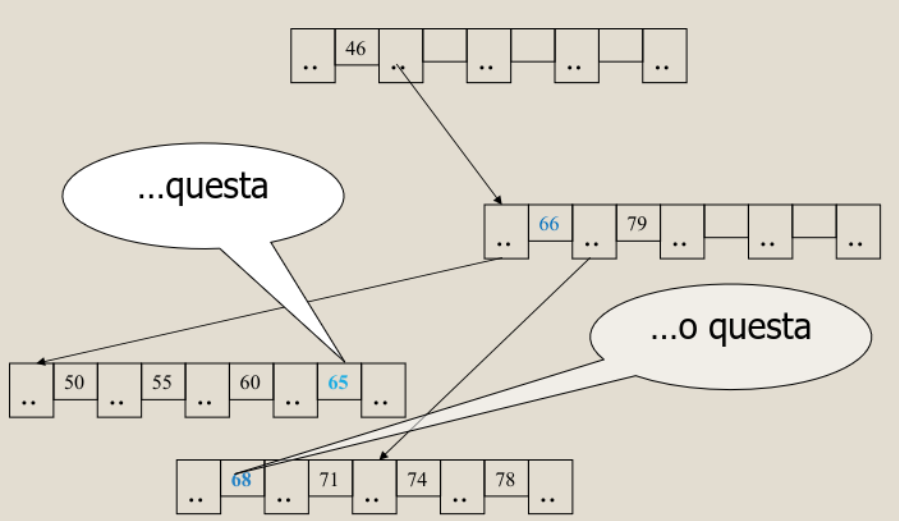




Cancellazione da B-tree di chiave K

0. La chiave k da cancellare sia nel nodo n
1. Se n è una foglia, si cancella k da n e si va al passo 3
2. Se n NON è una foglia si sostituisce k prendendo il più piccolo (grande) fra gli elementi maggiori (minori) di k; tale chiave sostitutiva è in una fogli f, per costruzione dell'albero, e quindi le va applicato il punto 1.
3. Controllo il numero di chiavi nella foglia:
   - Se $\lceil m/2 \rceil - 1 \leq$ *numero chiavi* $\leq m - 1$ allora fine cancellazione
   - altrimenti bisogna concatenare o bilanciare

Figure 6: Cancellazione della chiave 66



Segue un esempio di cancellazione con concatenazione:

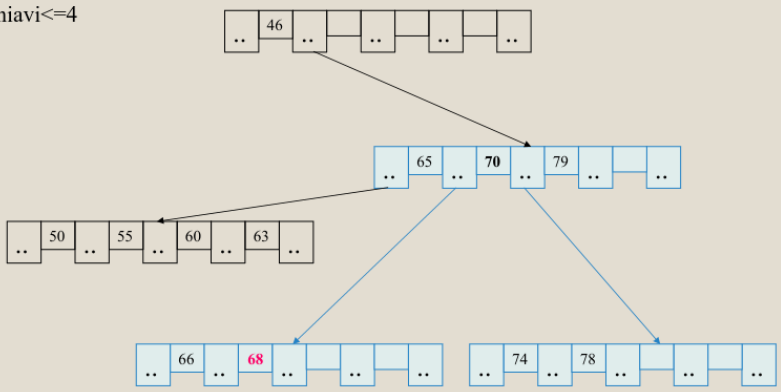
Cancellazione in B-tree: concatenazione

B-tree di ordine 5 (m=5)

$\lceil m/2 \rceil - 1 <= \text{num. chiavi} <= m-1$

2 <=num. chiavi<=4

Cancellazione della chiave 68



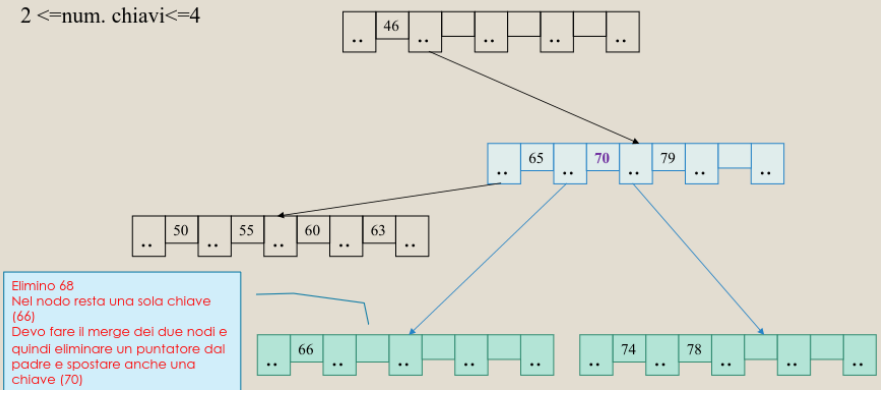
Cancellazione in B-tree: concatenazione

B-tree di ordine 5 (m=5)

$\lceil m/2 \rceil - 1 <= \text{num. chiavi} <= m-1$

2 <=num. chiavi<=4

Cancellazione della chiave 68



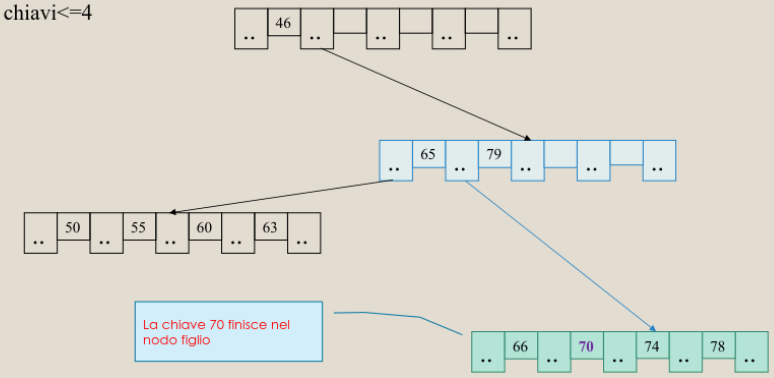
Cancellazione in B-tree: concatenazione

B-tree di ordine 5 (m=5)

$\lceil m/2 \rceil - 1 <= \text{num. chiavi} <= m-1$

2 <=num. chiavi<=4

Cancellazione della chiave 68



Esercizio *Dato il seguente B-Tree (Fan-Out = 4) eseguire le operazioni indicate mantenendo l'albero bilanciato:* B-Tree di ordine 4 (m = 4), quindi $1 \leq num\ Chiavi \leq 3$.

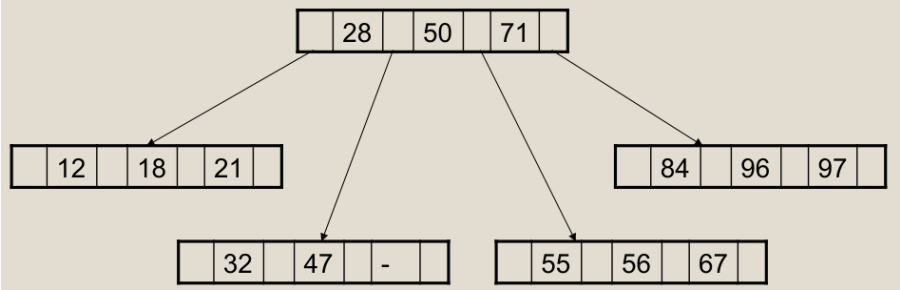


Figure 7: **Inserimento di k = 48**, inserimento a nodo NON saturo

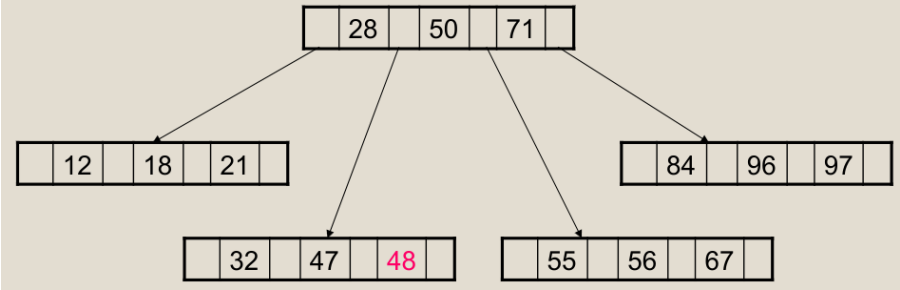


Figure 8: **Inserimento di k = 8** (inserimento a nodo saturo) 8 -12 -18 -21 Necessario SPLIT del nodo

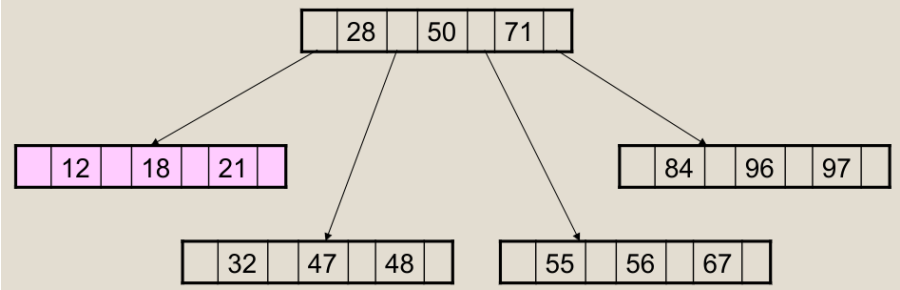


Figure 9: Suddivisione dell'informazione 8 – 12 – 18 – 21 in due 8 – 12 e 21 Aggiungiamo 18 al padre, facendo crescere di uno il numero dei puntatori → SPLIT del padre (18 – 28 – 50 – 71 in 18 – 28 e 71 più 50 nel padre)

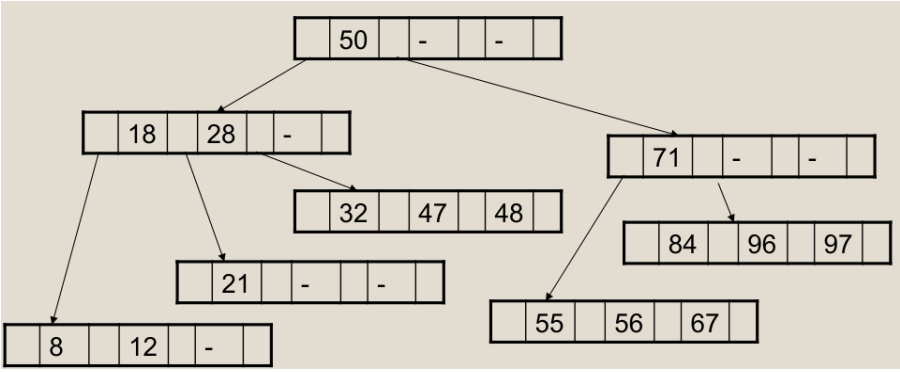


Figure 10: **Rimozione di k = 56**

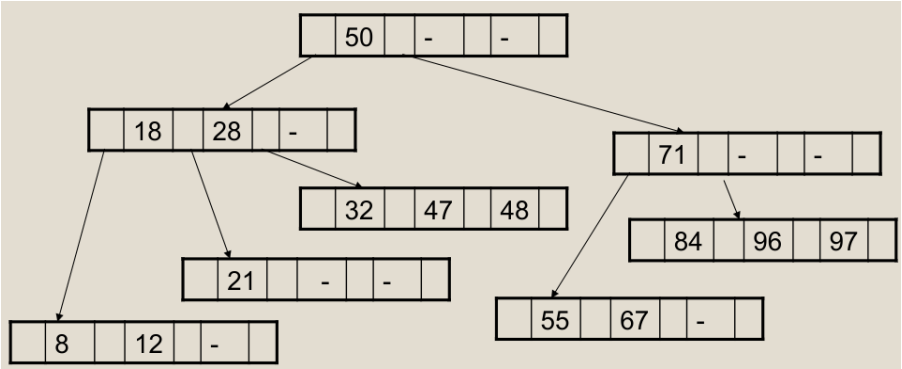


Figure 11: **Rimozione di k = 28**

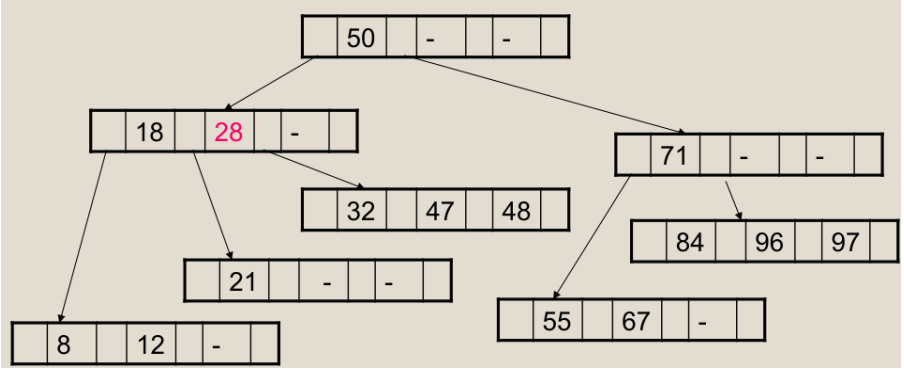


Figure 12: Bilanciamento della rimozione

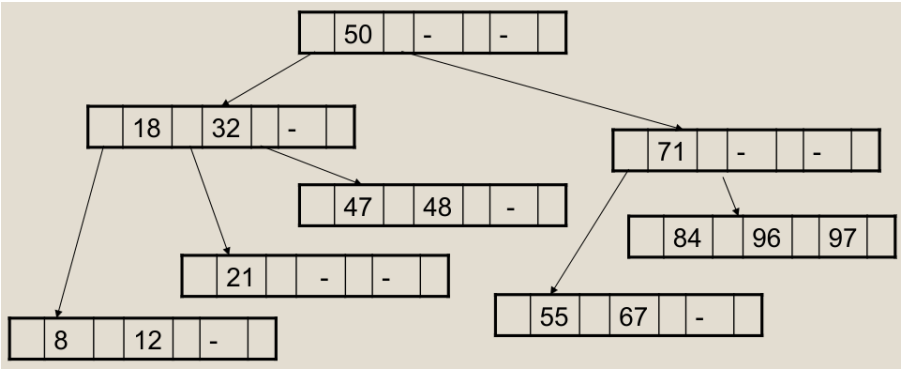


Figure 13: **Rimozione di k = 12**

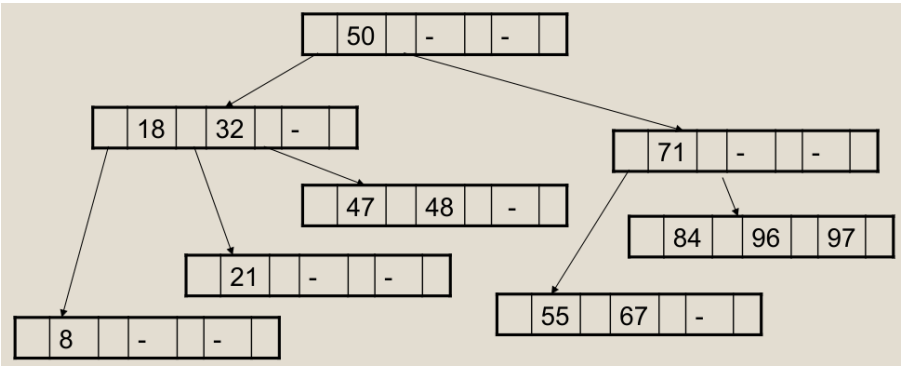


Figure 14: **Rimozione di k = 21**

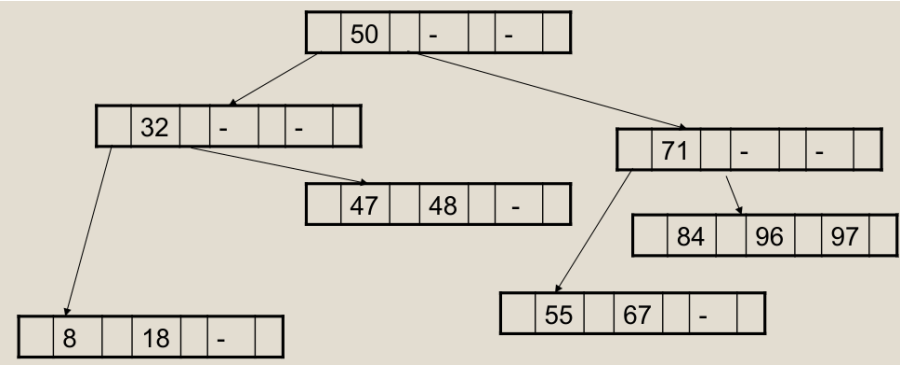


Figure 15: **Rimozione di k = 71**

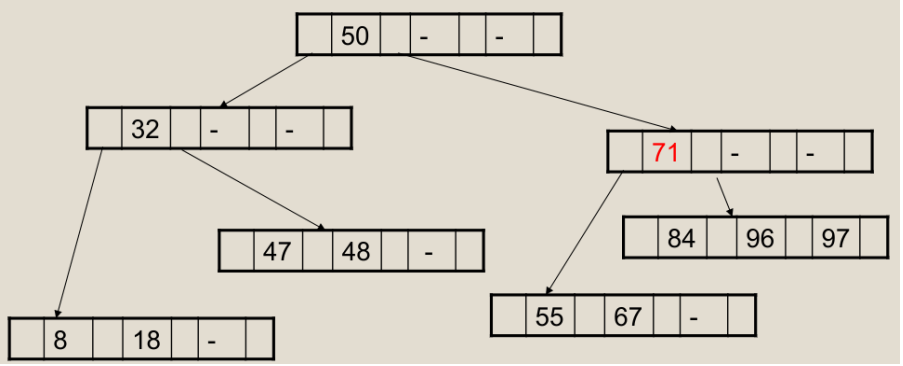


Figure 16: Passaggio intermedio

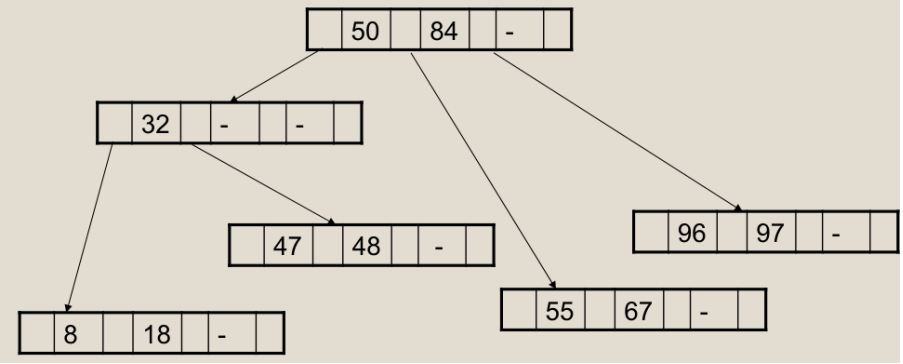
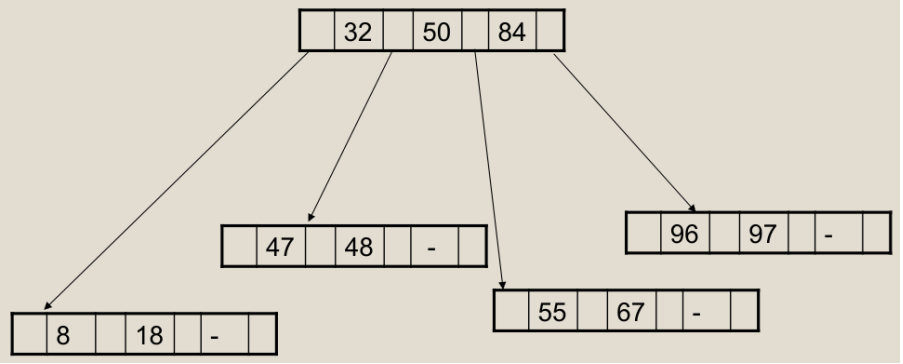
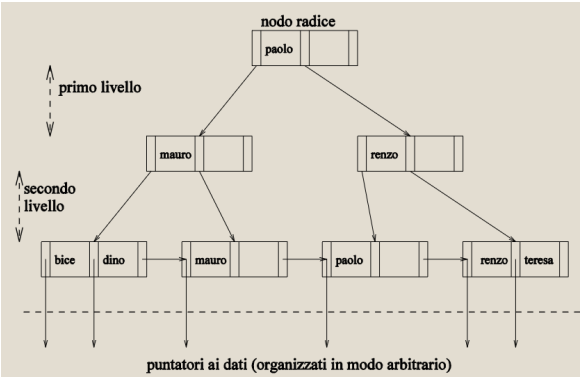


Figure 17: Bilanciamento

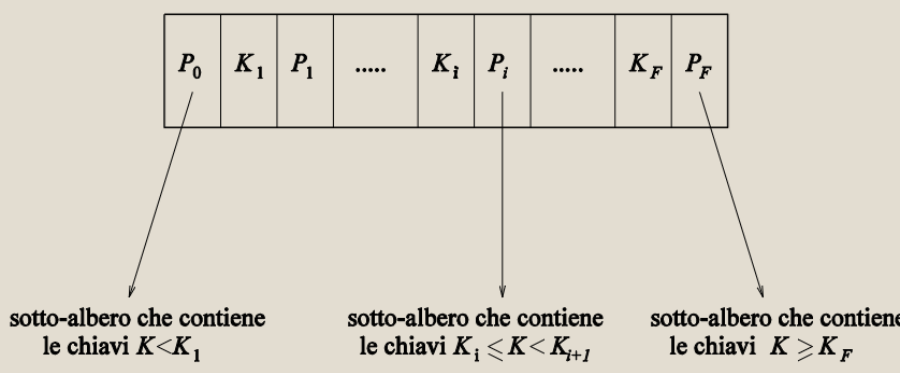


2.4.4 B+ Tree

- Le **foglie** sono **collegate in una lista**
- I puntatori ai dati sono **solo** nelle foglie
- L'inserimento delle chiavi avviene nei nodi foglia
- Ottimi per le ricerche su intervalli
- Molto usati nei DBMS



Organizzazione dei nodi:



Inserimento di una chiave Non provoca problemi quando la nuova chiave si può inserire in una *foglia non satura*. Quando la foglia non ha spazio disponibile si esegue lo SPLIT della foglila:

- Si suddivide l'informazione presente nella foglia e la nuoa informazione in due (in modo equilibrato) allocando due foglie al posto di una.
- Il valore mediano delle chiavi si **si inserisce anche in copia nel padre**

Lo SPLIT causa il crescere del numero dei puntatori al livello superiore. Lo SPLIT può dover essere ripetuto sul nodo padre. Lo SPLIT del padre (nodo non foglia) viene gestito come per i BTree:

- Si suddivide l'informazione in due, mettendo le chiavi maggiori del valore mediano nel nuovo nodo
- Il valore mediano delle chiavi viene spostato nel padre

Rimozione di una chiave Se la rimozione della chiave lascia due nodi foglia con spazio inutilizzato da consentire che tutta l'informazione in esse presente possa essere concentrata in una sola foglia si effettua un'operazione di **merge**, che richiede la modifica della disposizione dei puntatori. **MERGE** causa il decrescere del numero dei puntatori al livello superiore, e può dover essere ripetuto sul nodo padre.

Bilanciamento: se le modifiche causano differenze tra le lunghezze dei cammini tra radice e foglie è opportuo ri-bilanciare l'albero.

Esercizio B+Tree - Fan-Out = 4

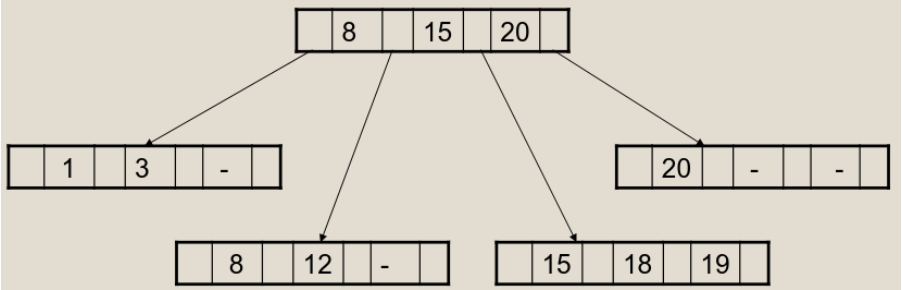


Figure 18: **Inserimento k = 5**

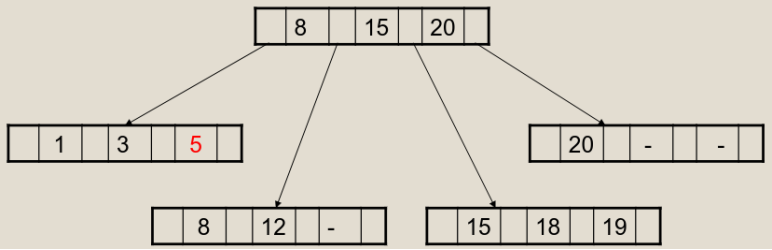


Figure 19: **Rimozione di k = 8**

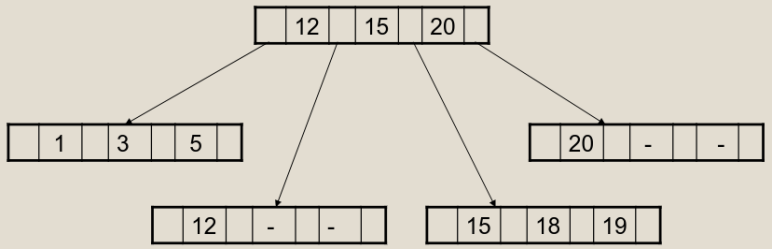


Figure 20: **Rimozione di k = 20**

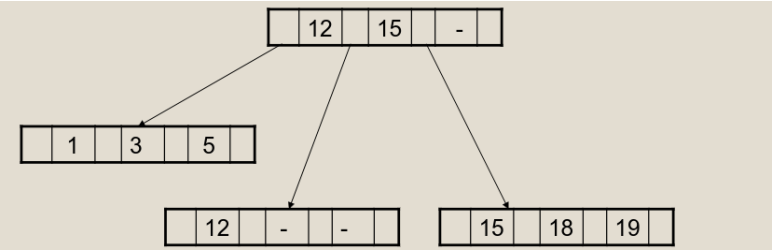


Figure 21: **Inserimento di k = 2**. Suddivisione dell'informazione 1 – 2 – 3 – 5 in due 1 – 2 e 3 – 5. Inseriamo 3 anche nel padre, facendo crescere di uno il numero dei puntatori

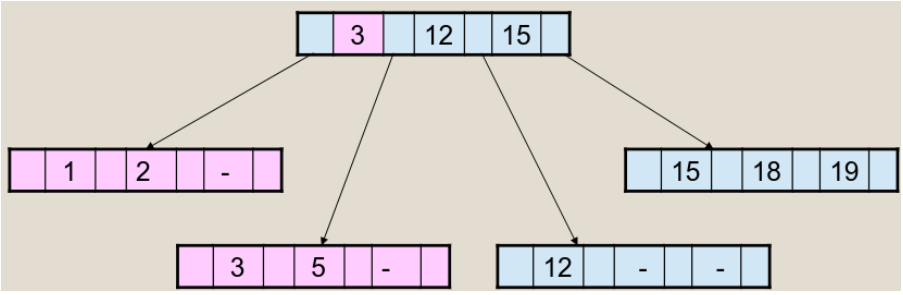
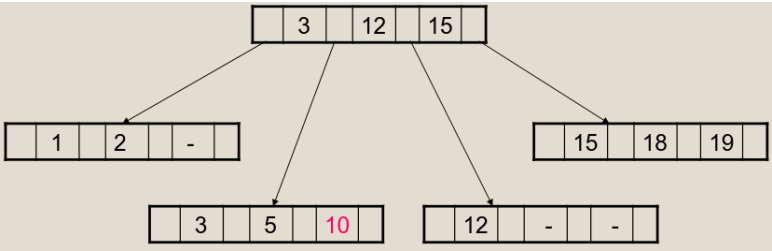


Figure 22: **Inserimento di k = 10**



Con l'inserimento di k = 6 abbiamo un inserimento in foglia satura con SPLIT ripetuto sul padre, è quindi necessario fare il primo SPLIT della foglia con 3 - 5 - 6 - 10.

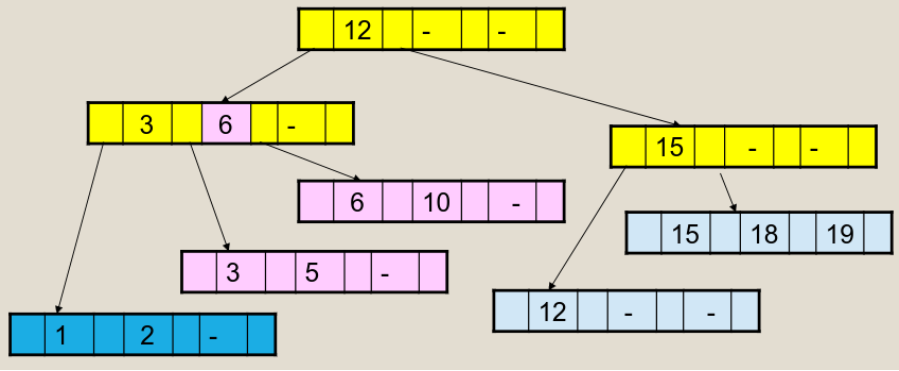


Figure 23: Suddivisione dell'informazione 3 – 5 – 6 – 10 in due 3 – 5 e 6 – 10 Inseriamo 6 anche nel padre → Chiavi nel padre 3 – 6 – 12 – 15 → SPLIT del padre Il padre non è foglia, quindi lo SPLIT viene gestito come nei B Tree → 3 – 6 – 12 – 15 in 3 – 6 e 15 e spostiamo 12 nel padre

Figure 24: **Rimozione di k = 2**

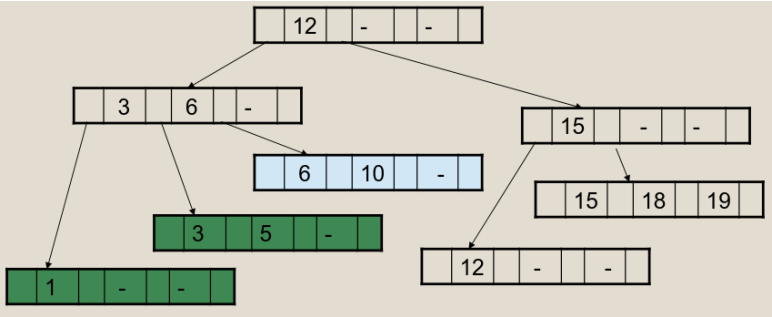
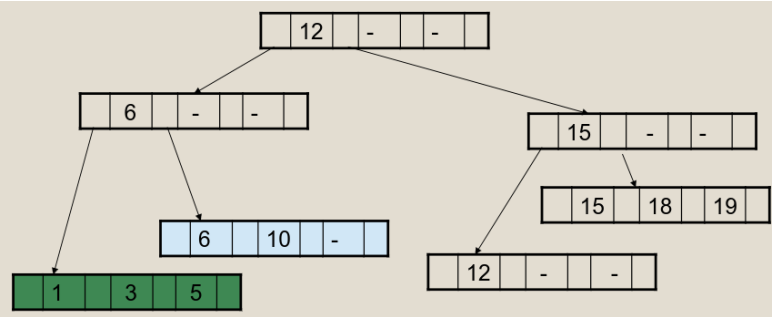


Figure 25: Fusione



Esercizio costruzione B+Tree Data la seguente lisa di valori chiave $L = (1, 6, 2, 9, 4, 7, 11, 8, 13)$:

1. Costruire un possibile B+Tree (fan-out = 4) che contenga i seguenti nodi foglia: $(1, 2, 4), (6, 7), (8, 9), (11, 13)$
2. Mostrare l'albero dopo l'inserimento del valore chiave 3

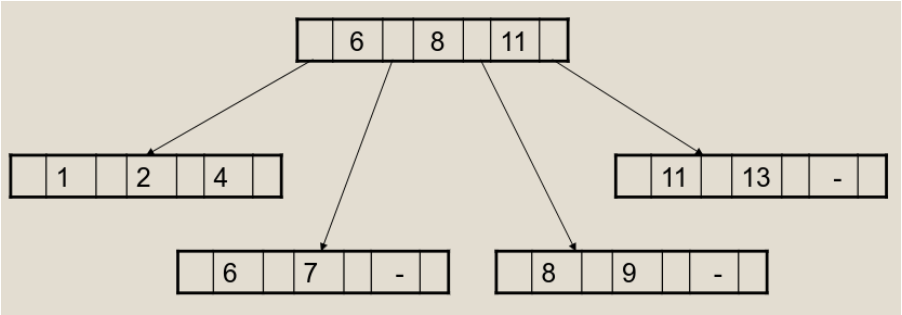
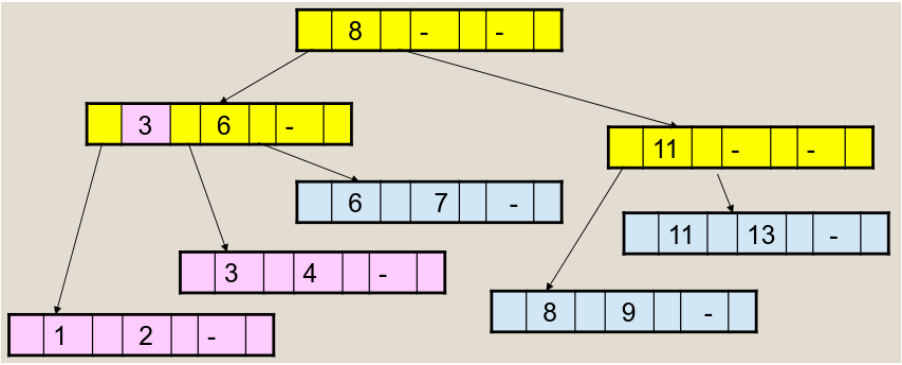


Figure 26: N.B. non rappresentiamo i collegamenti tra le foglie

Per l'inserimento di k = 3 è necessario lo SPLIT della foglia e del suo nodo padre.

Figure 27: **Insermento di k = 3** Suddivisione dell'informazione della foglia 1 – 2 – 3 – 4 in due 1 – 2 e 3 – 4 Aggiungiamo 3 al padre, facendo crescere di uno il numero dei puntatori → SPLIT del padre (nodo non foglia) → 3 – 6 – 8 – 11 in due 3 – 6 e 11 e spostiamo 8 nel padre



Esercizio 2 da risolvere: Data la seguente lista di valori chiave $L = (1, 2, 4, 7, 10, 11, 15, 16, 19, 20)$

1. Costruire un possibile B+Tree con fan-out = 4 che contenga i seguenti nodi foglia: $(1, 2, 4), (7, 10), (11, 15), (16, 19, 20)$
2. Mostrare l'albero dopo l rimozione del valore chiave *2*
3. Mostrare l'albero dopo l'inserimento del valore chiave 18.

3 Gestione delle transazioni

Riguardano aspetti tecnologici che caratterizzano il funzionamento dei DBMS riguardo la modalità con cui viene garantita la possibilità di conservare a lungo termine e in modo corretto il contenuto delle BD anche in presenza malfunzionamenti e guasti e di accessi concorrenti.

Transazione: Identifica una unità di lavoro elementare svolta da un'applicazione.
Caratteristiche di:

- **Correttezza**: La correttezza garantisce che il database passi da uno stato consistente a un altro stato consistente, rispettando tutti i vincoli di integrità (chiavi, vincoli di dominio, ecc.).
- **Robustezza**: La robustezza riguarda la capacità del sistema di mantenere le modifiche apportate dalle transazioni confermate (commit), anche in presenza di guasti hardware o software (es. blackout).
- **Isolamento**:L'isolamento assicura che transazioni concorrenti (eseguite contemporaneamente da più utenti) non interferiscano tra loro, comportandosi come se venissero eseguite in modo sequenziale.

Caratterizzata da:

- **Inizio** *begin-transaction / start transaction*
- **Fine** *end-transaction*

al cui interno deve essere eseguito una e una sola volta uno dei seguenti comandi:

- *commit work*: per terminare correttamente.
- *rollback work*: per abortire la transazione.

Un sistema transazionale (On-Line Transaction Processing) è in grado di definire ed eseguire transazioni per conto di un certo numero di applicazioni concorrenti.

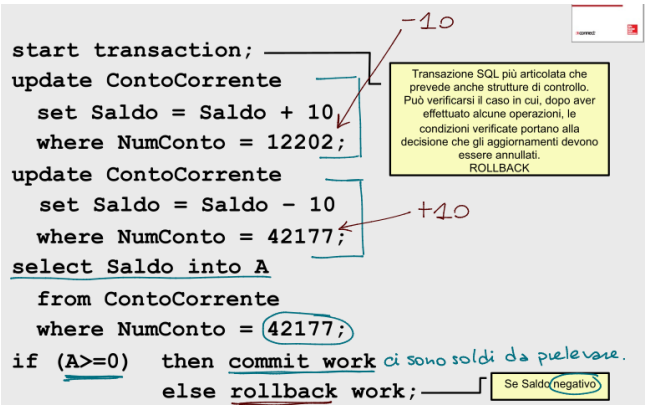


Figure 28: Esempio di transazione con varie decisioni

3.1 Proprietà ACID delle transazioni

Il codice che viene eseguito all'interno di una transazione gode di proprietà dette **ACIDE**.

3.1.1 Atomicità

Una transazione è un'unità atomica (indivisibile) di elaborazione: o vengono resi visibili tutti gli effetti di una transazione oppure la transazione non deve avere alcun effetto. → **Approccio TUTTO o NIENTE**. Non può lasciare la base di dati in uno stato intermedio:

- un guasto o un errore *prima del commit* debbono causare l'annullamento (**UNDO**) delle operazioni svolte.
- un guasto o errore dopo il commit non deve avere conseguenze, se necessario vanno ripetute (**REDO**) le operazioni.

Esito:

- **Commit** = caso normale e più frequente.
- **Abort o rollback**: può essere richiesto dall'applicazione (suicidio) o può essere richiesto dal sistema per violazione dei vincoli, concorrenza, incertezza in caso di fallimento (omicidio).

3.1.2 Consistenza

La consistenza richiede che l'esecuzione della transazione non violi i vincoli di integrità definiti sulla BD. Quando il sistema rileva che una transazione sta violando uno dei vincoli (es. inserimento di tupla con chiave già presente) il sistema interviene per annullare la transazione o correggere la violazione del vincolo. La verifica dei vincoli di integrità può essere

- Immediata (nel corso della transazione)
- Differita (alla conclusione della transazione)

La consistenza garantisce che se lo stato iniziale è corretto anche lo stato finale è corretto,

3.1.3 Isolamento

L'isolamento richiede che l'esecuzione di una transazione sia indipendente dall'esecuzione contemporanea di altre transazioni. La transazione non risente degli effetti delle altre transazioni concorrenti: l'esecuzione concorrente di una collezione di transazioni deve **produrre un risultato che si potrebbe ottenere con una esecuzione sequenziale**.
Conseguenza: Una transazione non espone i suoi stati intermedi. Si evita l' "effetto domino".

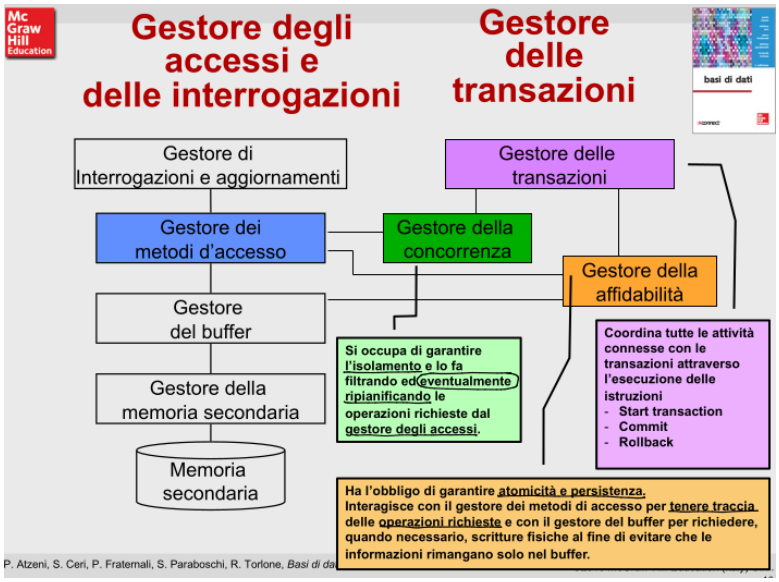
3.1.4 Durabilità (persistenza)

Gli effetti di una transazione andata in commit non vanno perduti ("durano per sempre"), anche in presenza di guasti. (Commit significa impegno). Un DBMS deve garantire che nessun dato venga perso per nessun motivo.

3.2 Gestore delle transazioni

Per ciascuna delle proprietà sopra elencate, vi sono dei moduli che verificano che siano applicate:

- **Atomicità e durabilità**: Gestore dell'affidabilità (Reliability manager)
- **Isolamento**: Gestore della concorrenza
- **Consistenza**: La CONSISTENZA è gestita dai compilatori del DDL che introducono controlli sui dati e procedure di verifica eseguite dalle transazioni.



Domanda d'esame: come gestisce il DBMS le transazioni?

3.2.1 Gestore dell'affidabilità

Assicura **atomicità e durabilità** (persistenza). Gestisce l'esecuzione dei comandi transazionali: *start transaction (B, begin), commit work (C), rollback work (A, abort)* e le operazioni di ripristino (recovery) dopo i guasti: *ripresa a caldo e ripresa a freddo (warm restart e cold restart)*. Usa il log: un archivio permanente che registra le operazioni svolte dal DBMS. Ogni azione di scrittura sulla BD viene protetta tramite una azione sul LOG.

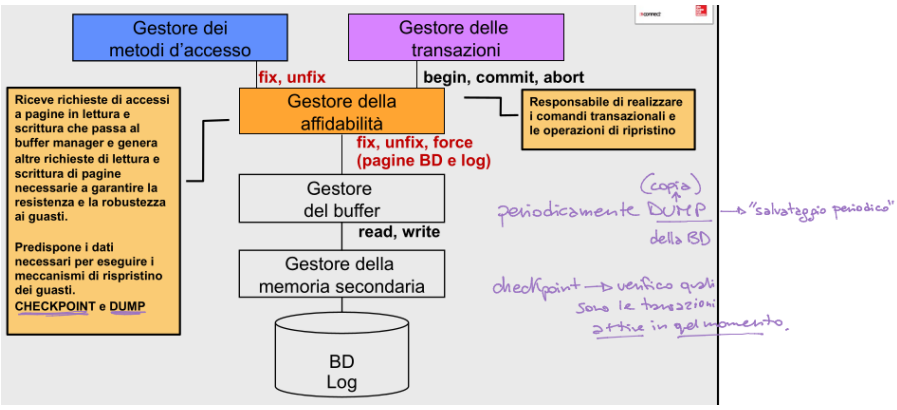


Figure 29: Architettura del gestore dell'affidabilità

Il gestore dell'affidabilità deve disporre di **MEMORIA STABILE** cioè di memoria che risulti resistente ai guasti:

- Memoria centrale: non è persistente
- Memoria di massa: è persistente ma può danneggiarsi
- Memoria stabile: memoria che non può danneggiarsi

è una **astrazione**, nessuna memoria può avere probabilità nulla di fallimento, viene perseguita attraverso la *ridondanza:* dischi replicati, nastri, ... Un guasto della memoria stabile è considerato catastrofico

3.2.2 Il log

Domanda d'esame: cos'è il log e cosa vi è contenuto dentro?
Il log è un file sequenziale gestito dal gestore dell'affidabilità, scritto in memoria stabile. Permette di ricostruire il contenuto della BD a seguito di guasti. "Diario di bordo": riporta tutte le operazioni in ordine.
Record nel log:

- **Operazioni delle transazioni:** Registrano le attività svolte da ciascuna transazione nell'ordine in cui vengono effettuate.
  - ◇ *begin, B(T)*
  - ◇ *insert, I(T,O,AS)*
  - ◇ *delete, D(T,O,BS)*
  - ◇ *update, U(T,O,BS,AS)*
  - ◇ *commit, C(T), abort, A(T)*

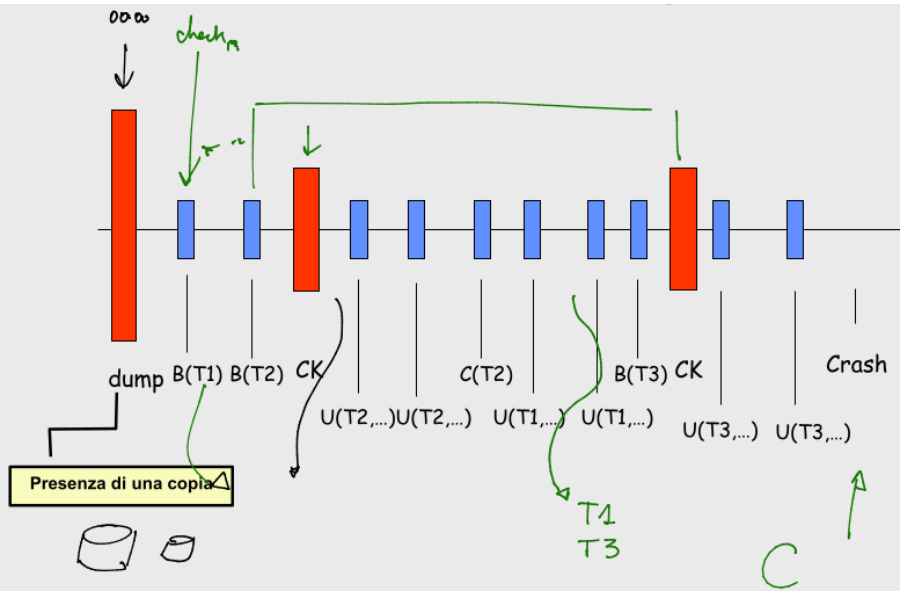
  Dove

  - ◇ T = id della transazione
  - ◇ O = id oggetto coinvolto
  - ◇ AS = valore dell'oggetto dopo (AFTER) la modifica
  - ◇ BS = valore dell'oggetti prima (BEFORE) della modifica

- **Record di sistema:** Indicano l'effettuazione di operazioni specifiche del gestore dell'affidabilità:

- ◇ *dump*: Copia completa e consistente della BD (effettuata quando il sistema non è operativo) = BACKUP
- ◇ *checkpoint*: Operazione che viene svolta periodicamente ed ha l'obiettivo di registrare quali transazione sono attive

Il log serve "a ricostruire" le operazioni: checkpoint e dump servono ad evitare che la ricostruzione debba partire dall'inizio dei tempi si usano con riferimento a tipi di guasti diversi.



3.2.3 UNDO e REDO

Operazioni specifiche per disfare e rifare le operazioni sulla BD, sono di competenza del gestore dell'affidabilità.

- **UNDO** di una azione su un **oggetto O**:
  - ◇ *Update, Delete*: ricopiare il valore del **before state (BS)** nell'oggetto O
  - ◇ *Insert:* eliminare O
- **REDO** di una azione su un **oggetto O**:
  - ◇ *Update, Delete:* Ricopiare il valore dell'after state (AS) nell'oggetto O
  - ◇ *Delete:* Eliminare O

Idempotenza di Undo e Redo:

- *undo(undo(A)) = undo(A)*
- *redo(redo(A)) = redo(A)*

L'effettuazione di un numero arbitrario di UNDO e REDO della stessa azione equivale allo svolgimenti di tale azione una sola volta. Questa proprietà è utile nel caso in cui si abbiano errori durante le operazioni di ripristino e sia necessario ripetere UNDO e REDO.

3.2.4 Checkpoint

Operazione che serve a "fare il punto" della situazione, semplificando le successive operazioni di ripristino: ha lo scopo di registrare quali transazioni sono attive in un certo istante e dualmente di confermare che le altre o non sono iniziate o sono finite. Vi sono varie modalità, vediamo la più semplice:

- Si sospende l'accettazione di richieste di ogni tipo (scrittura, inserimenti, ..., commit, abort)
- Si trasferiscono in memoria di massa (tramite force) tutte le pagine sporche relative a transazioni andate in commit
- Si registrano sul log in modo sincrono (force) gli identificatori delle transazioni in corso

- Si riprende l'accettazione delle operazioni

Così siamo sicuri che

- Per tutte le transazioni che hanno effettuato il commit i dati sono in memoria di massa, quindi registrate nella BD in modo permanente
- Le transazioni "a metà strada" (non hanno ancora fatto COMMIT o ABORT) sono elencate nel checkpoint

3.2.5 Dump

Copia completa ("di riserva", backup) della base di dati

- Solitamente prodotta mentre il sistema non è operativo
- Salvato in memoria stabile (es. Su nastro)
- Un record di dump nel log indica il momento in cui il log è stato effettuato (e dettagli pratici, file, dispositivo, ...)

3.2.6 Esito di una transazione

L'esito di una transazione è determinato irrevocabilmente quando viene scritto il **record di commit nel log in modo sincrono**, con una **force**. Un guasto prima di tale istante porta ad un UNDO di tutte le azioni, per ricostruire lo stato originario della base di dati. Un guasto successivo non deve avere conseguenze: lo stato finale della base di dati deve essere ricostruito, con redo se necessario. Record di abort possono essere scritti in modo asincrono.

3.2.7 Regole fondamentali per il log

Durante il funzionamento normale delle transazioni il gestore dell'affidabilità deve garantire che siano seguite due regole; le regole definiscono i requisiti minimi che consentono di ripristinare la correttezza della BD a fronte di guasti:

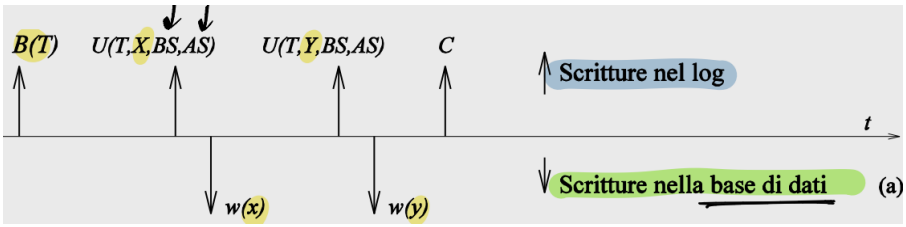
- **Write-Ahead-Log (WAL):** *Si scrive il giornale (parte before) prima del database.*

  Impone che la parte BEFORE STATE dei record di un Log venga scritta nel Log PRIMA di effettuare la corrispondente operazione sulla BD. Consente di disfare le scritture già effettuate in memoria di massa da parte di una transazione che non ha ancora effettuato un COMMIT. Per ogni aggiornamento viene reso disponibile in modo affidabile il valore precedente la scrittura.

  Versione semplificata: Scrivere prima record di Log e poi nel DB, scrivi il Log per primo.

  Scrive prima il record di Log e poi il record nella BD. Al COMMIT tutte le pagine della BD modificate dalla transazione sono scritte in memoria di massa. **Non richiede REDO**.

  Tecnica di aggiornamento immediato, permette comunque il recovery: se una transazione fallisce prima del COMMIT **l'effetto di eventuali operazioni effettuate deve essere annullato**.

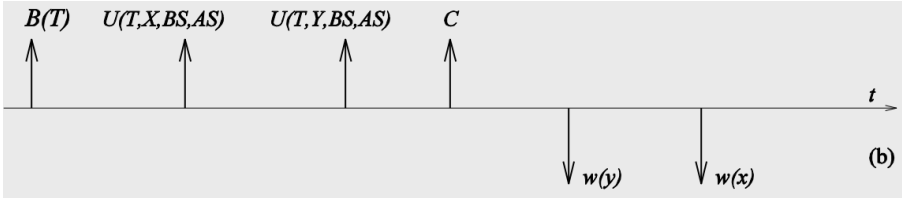


- **Commit-Precedenza:** Si scrive il giornale (parte after) prima del commit.

  Impone che la parte AFTER STATE dei record di un Log venga scritta nel Log PRIMA di effettuare il COMMIT. Consente di rifare le scritture già decise da una transazione che ha effettuato il COMMIT ma le cui pagine modificate non sono ancora state trascritte dal *Buffer Manager* in memoria di massa.

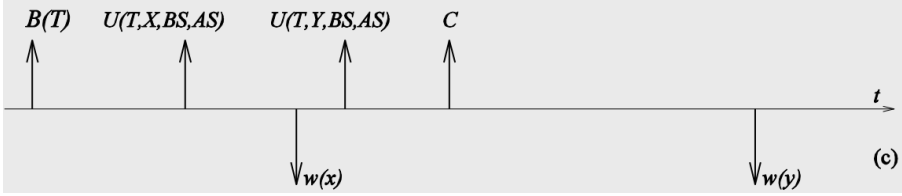
  Versione semplificata: Record di Log scritti prima del COMMIT.

  La scrittura dei record di Log precede le azioni sulla BD. Le azioni sulla BD avvengono dopo il COMMIT e la scrittura del record di COMMIT nel Log. **Non richiede UNDO** Tecnica di aggiornamento differito: i dati non sono fisicamente aggiornati fino all'esecuzione del COMMIT. **Se la transazione fallisce non è necessario l'UNDO**.



- Più generale e comunemente utilizzato è un **compromesso** tra le due regole sopra scritte: Le scritture sulla BD, una volta protette dalle opportune scritture sul Log, possono avvenire in un qualunque momento rispetto alla scrittura del record di COMMIT nel Log. Consente al Buffer manager di ottimizzare le operazioni.

  Richiede sia UNDO che REDO.

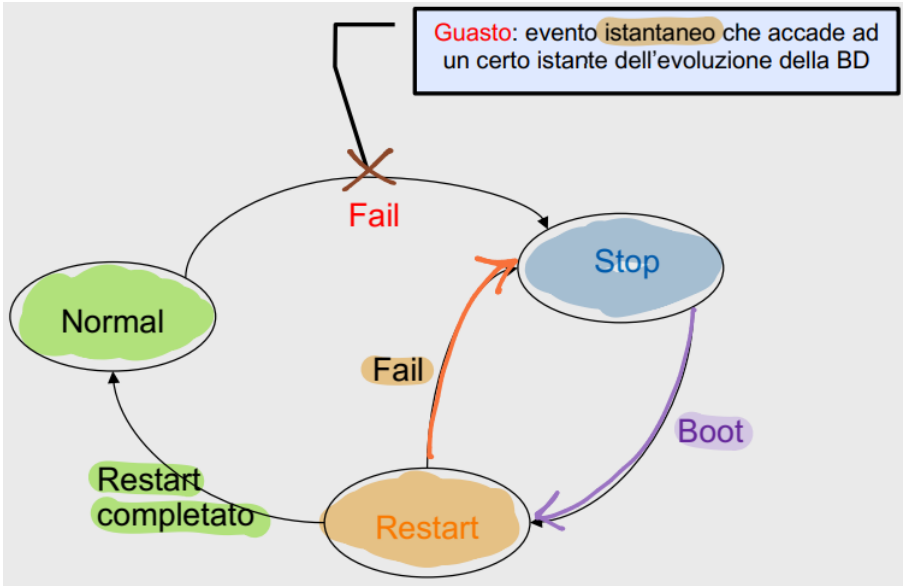


4 Gestione dei guasti

I guasti possono essere classificati in:

- **Guasti di Sistema (Guasti "soft")**: Errori di programma, crash di sistema, caduta di tensione. Si perde la memoria centrale (buffer), ma *non si perde la memoria secondaria* (BD e Log). Richiede la **ripresa a caldo (warm restart)**.
- **Guasti di Dispositivo (Guasti "hard")**: Guasti sui dispositivi di memoria secondaria, si perde anche la memoria secondaria (BD), ma *non si perde la memoria stabile* (e quindi il log). Richiede la **ripresa a freddo (cold restart)**.

Quando il Sistema individua un guasto (sia di Sistema che di dispositivo) immediatamente **forza un arresto** completo delle transazioni, e il successivo ripristino del corretto funzionamento del Sistema Operativo (**Boot**). Quindi viene attivata una procedura di ripresa *A CALDO* se il guasto è di sistema, *A FREDDO* se il guasto è di dispositivo. Al termine della procedura di ripresa, il sistema è di nuovo utilizzabile. Questo viene chiamato **Modello "fail-stop"**.



Obiettivi della procedura di ripresa: Classificare le transazioni in

- **Completate** (tutti i dati in memoria stabile)

- **In commit** ma non necessariamente completate: Può servire REDO per rifare le azioni e garantire la consistenza
- **Senza commit:** Vanno annullate con UNDO per lasciare la BD nel suo stato iniziale precedente alla esecuzione della transazione

Quest'ultime due sono transazioni potenzialmente attive: si classificano in base alla informazione presente nel Log.

4.1 Ripresa a caldo

Garantisce l'atomicità e la persistenza delle transazioni.

- **Atomicità**: viene garantito che le transazioni in corso al momento del guasto lascino la BD nello stato iniziale o nello stato finale.
- **Persistenza**: per le transazioni completate ma non ancora trascritte in memoria di massa si prevede la scritture in memoria di massa

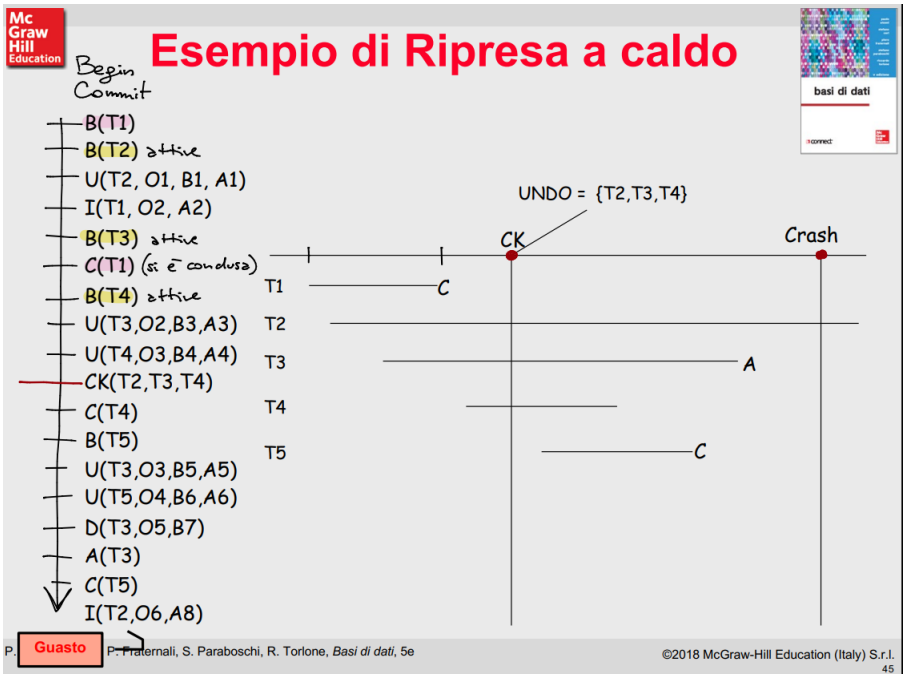
È divisa in quattro fasi:

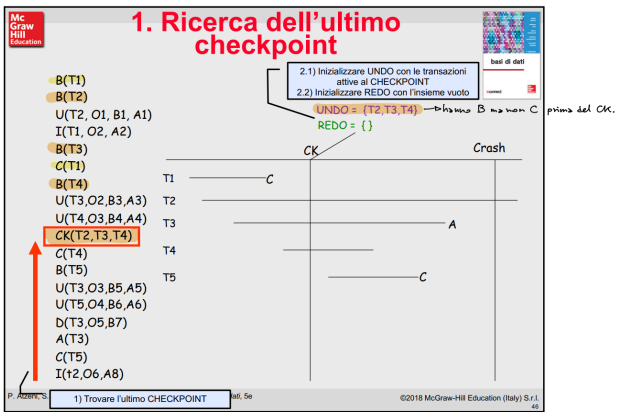
1. Trovare l'ultimo **CHECKPOINT** (ripercorrendo il Log a ritroso)
2. **Costruire gli insiemi UNDO (transazioni da disfare) e REDO (transazioni da rifare)**

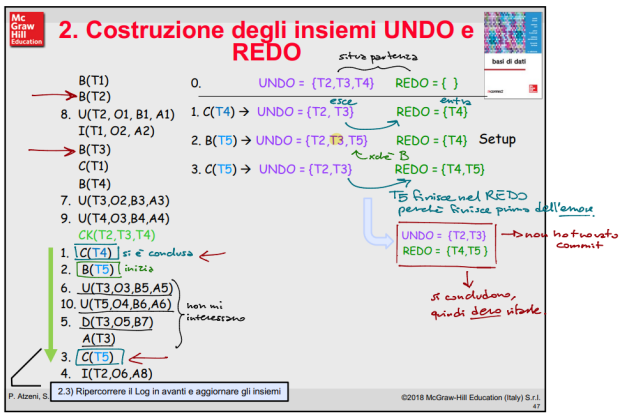
   Si inizializza l'insieme **UNDO** (transazioni da disfare) con le **transazioni attive al CHECKPOINT**. Si inizializza l'insieme REDO (transazioni da rifare) con l'insieme vuoto. Si percorrere il log in avanti

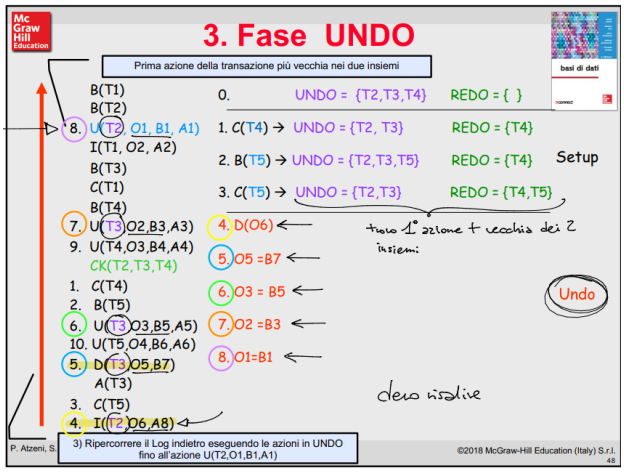
   - aggiungendo ad **UNDO tutte le transazioni per cui è presente un record di BEGIN**
   - e **spostando da UNDO a REDO le transazioni per cui è presente il record di COMMIT**
3. Ripercorrere il log all'indietro, fino alla più vecchia azione delle transazioni in UNDO e REDO, disfacendo tutte le azioni delle transazioni in UNDO – Questa azione potrebbe precedere il record di CHECKPOINT
4. Ripercorrere il log in avanti, rifacendo tutte le azioni delle transazioni in REDO, nell'ordine in cui sono presenti nel Log

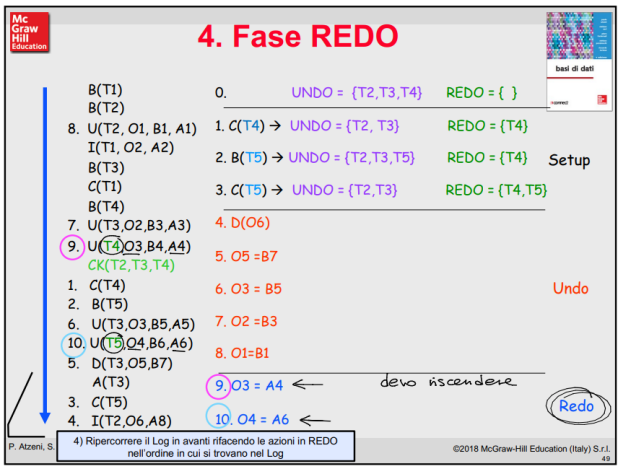
Esempio di ripresa a caldo:











4.2 Ripresa a freddo

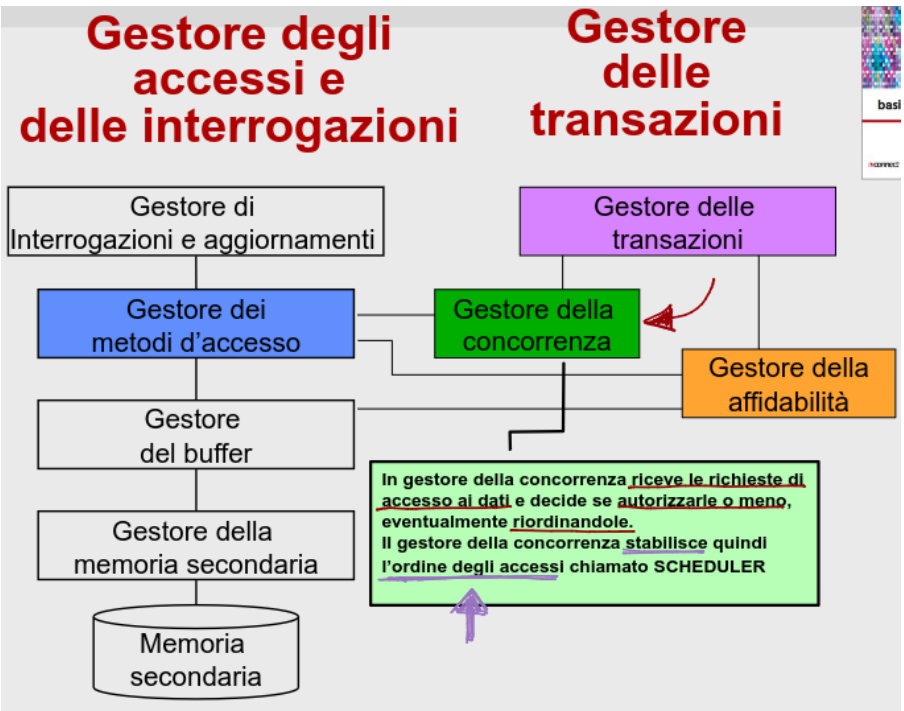
La ripresa a freddo risponde ad un guasto che provoca il deterioramento di una parte della BD. È articolato in 3 fasi:

- Si ripristinano i dati a partire dal backup (DUMP): Si accede al più recente record di DUMP nel Log
- Si **eseguono le operazioni registrate** sul Log fino all'istante del guasto: si ripercorre in avanti il Log, si eseguono sia le azioni sulla BD che le azioni di COMMIT e ABORT, riportandosi nella situazione precedente
- Si esegue una **ripresa a caldo**

Domanda d'esame: si enunci la differenza fra la ripresa a caldo e la ripresa a freddo.

5 Gestore della concorrenza

Un DBMS spesso deve servire diverse applicazioni e rispondere alle richieste di diversi utenti. Il numero di transazioni è molto elevato, quindi è indispensabile che le transazioni di un DBMS vengano eseguite CONCORRENTEMENTE, la loro esecuzione SERIALE è impensabile.
In gestore della concorrenza riceve le richieste di accesso ai dati e decide se autorizzarle o meno, eventualmente riordinandole. Il gestore della concorrenza stabilisce quindi l'ordine degli accessi chiamato **SCHEDULER**
Decine o centinaia di transazioni al secondo (Transactio Per Second - TPS **non possono essere seriali**. I TPS sono l'unità di misura utilizzata per caratterizzare il carico applicativo di un DBMS.
Domanda d'esame: si descrivano brevemente le anomalie delle transazioni concorrenti



5.1 Anomalie delle transazioni concorrenti

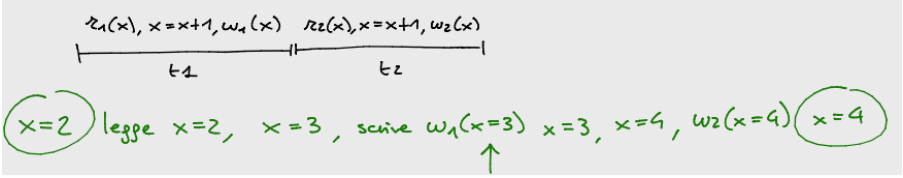
L'esecuzione concorrente delle transazioni può causare problemi di correttezza.

5.1.1 Perdita di aggiornamento

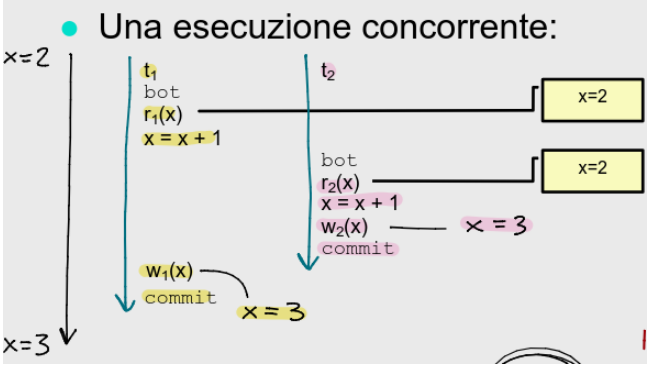
Due transazioni identiche che operano sullo stesso oggetto x della BD:

- $t_1 : r_1(x), x = x + 1, w_1(x)$ dove $r_1(x) \rightarrow$ lettura dell'oggetto x da parte della transazione t_1 e $w_1(x) \rightarrow$ lettura dell'oggetto x da parte della transazione t_1
- $t_2 : r_2(x), x = x + 1, w_2(x)$

Se ad esempio, inizialmente $x = 2$, dopo una esecuzione seriale x = 4. Viceversa, per una esecuzione concorrente

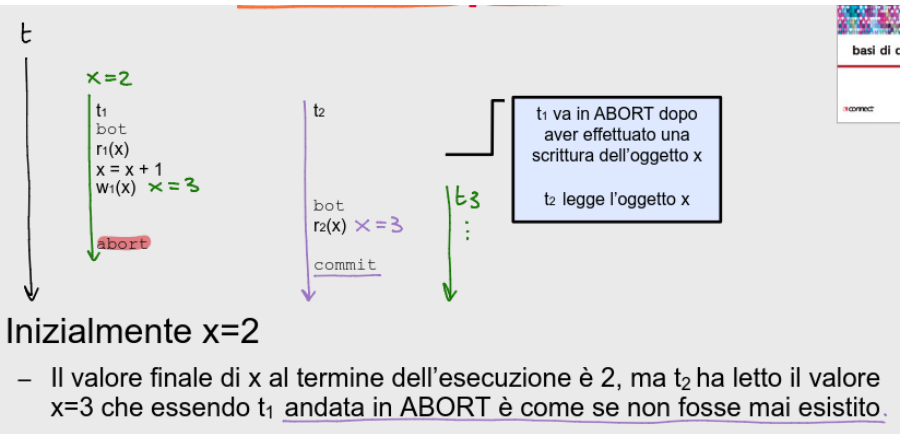


un aggiornamento viene perso, poiché alla fine di questa esecuzione $x = 3$: entrambe le transazioni leggono 2 come valore iniziale di x, gli effetti della transazione t_2 (la prima che scrive il nuovo valore di x) vengono persi.



5.1.2 Lettura sporca

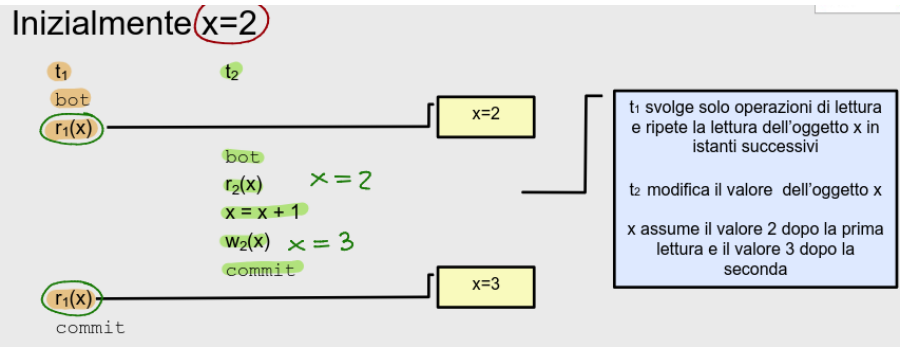
Aspetto critico: una transazione legge uno stato intermedio (sporco) e lo può comunicare all'esterno.



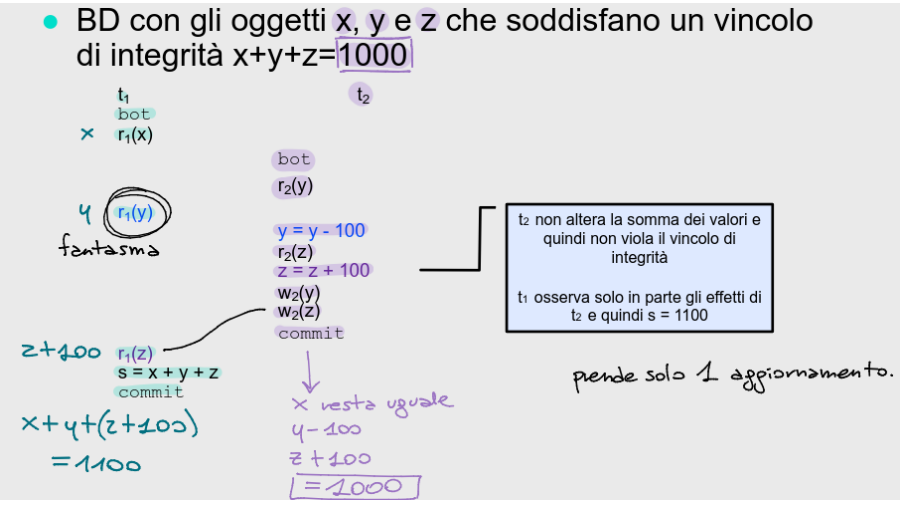
L'unico modo per ripristinare la correttezza a seguito dell'ABORT di t_1 sarebbe di **imporre l'ABORT** di t_2 e ricorsivamente di tutte le transazioni che avessero letto dati modificati da t_2, con un *effetto domino*; è una soluzione costosa da gestire e talvolta impraticabile, ad esempio se t_2 avesse comunicato il dato all'esterno. Quindi **le transazioni non devono esporre dati intermedi.**

5.1.3 Letture inconsistenti

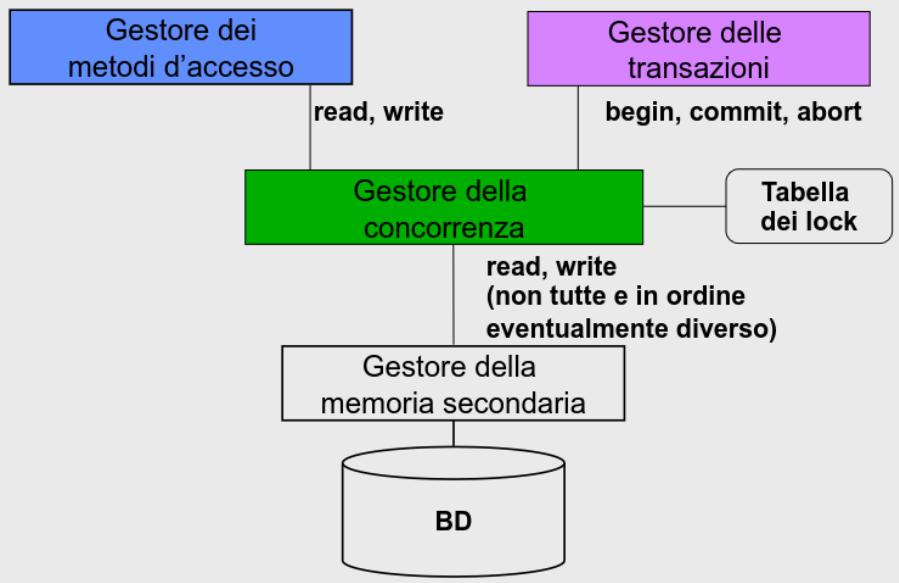
La transazione legge valori diversi per lo stesso oggetto x senza modificarlo. Sarebbe opportuno che una transazione che accede due volte alla BD trovi esattamente lo stesso valore per ciascun dato letto. **La transazione non deve risentire dell'effetto di altre transazioni**.



5.1.4 Aggiornamento fantasma



5.2 Controllo di Concorrenza



La garanzia dell'isolamento (quindi anche di non esporre stati intermedi) è molto onerosa. I DBMS prevedono la possibilità di specificare un grado di protezione ridotto: Alcune anomalie vengono evitate, altre anomalie possono manifestarsi. Si può rinunciare ad un controllo di concorrenza stringente per migliorare le prestazioni.

Modello formale di transazione una transazione è definita come una **sequenza di azioni** di **lettura e scrittura:** $t_1 = r_1(x), r_1(y), w_1(x), w_1(y)$. Ogni transazione è un oggetto sintattico di cui si conoscono soltanto le operazioni di ingresso e uscita. Le operazioni nello schedule compaiono seguendo l'ordine temporale con cui sono eseguite sulla BD. Il controllo di concorrenza ha la funzione di accettare alcuni schedule e di rifiutarne altri.

5.2.1 Soluzioni iniziali NON applicabili

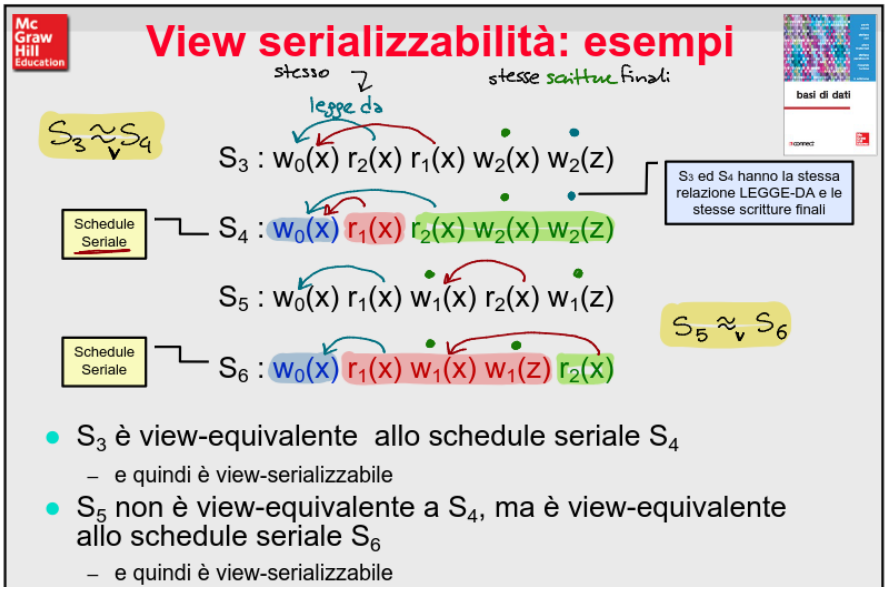
Ipotesi semplificativa: inizialmente assumeremo di sapere l'esito (COMMIT/ABORT) delle transazioni che appaiono in uno schedule. Potremo ignorare le transazioni che producono un ABORT, togliendo dallo schedule tutte le loro operazioni e ci concentreremo solo sulle transazioni che producono un COMMIT. In questo caso lo schedule si dice una **commit-proiezione**. Questa assunzione permette uno studio teorico, ma non è accettabile in pratica. Lo schedule va accettato o rifiutato senza conoscere l'esito finale delle transazioni coinvolte.
Idea base: individuare classi di schedule serializzabili che siano sottoclassi degli schedule possibili, siano serializzabili e la cui proprietà di serializzabilità sia verificabile a costo basso.

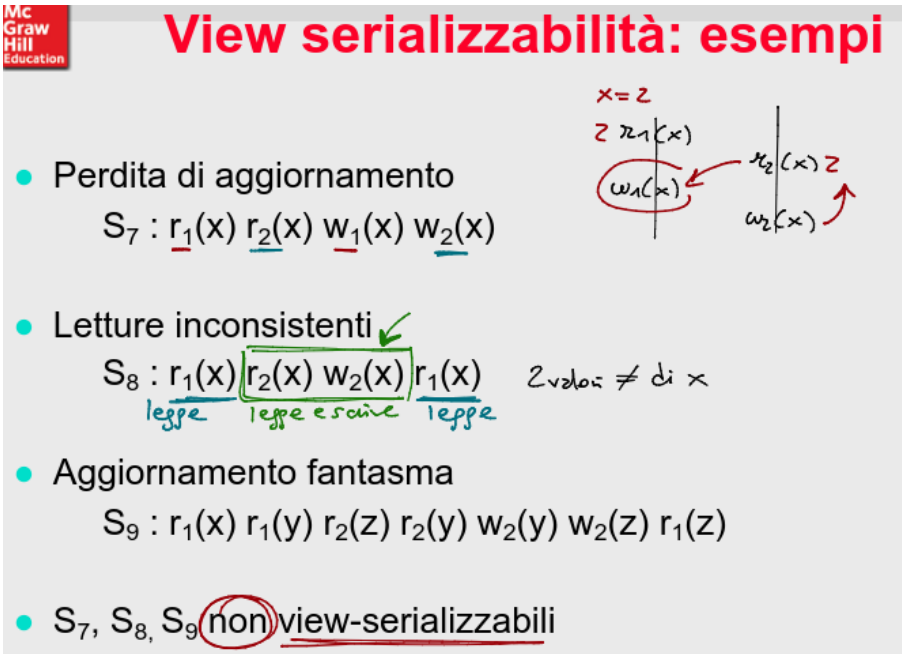
View-Equivalenza Nozione di equivalenza concettualmente interessante, ma inutilizzabile in pratica per ragioni computazionali.

- **LEGGE DA:** $r_i(x)$ **legge-da** $w_j(x)$ in uno schedule S, se $w_j(x)$ precede $r_i(x)$ in S e non c'è $w_k(x)$ fra $r_i(x)$ e $w_j(x)$ in S.
- **SCRITTUA FINALE:** $w_i(x)$ in uno schedule S è **scrittura finale** se è l'ultima scrittura dell'oggetto x in S.

Due schedule sono **view-equialenti** $S_i \approx_V S_j$ se hanno la **stessa relazione legge-da e le stesse scritture finali.**

View Serializzabilità Uno schedule è view-serializzabile se esiste uno schedule seriale view-equivalente ad esso. L'insieme degli schedule view-serializzabili è indicato con **VSR**.
Determinare la view-equivalenza di due schedule è un problema con complessità lineare. Determinare se uno schedule è view-equivalente ad un qualsiasi schedule seriale è un problema NP-difficile. Il concetto di view-equivalenza non è utilizzabile in pratica per caratterizzare la serializzabilità/.Si preferisce definire una condizione di equivalenza più ristretta, che non copra tutti i casi di equivalenza tra schedule, ma sia utilizzabile nella pratica perché ha una complessità inferiore.



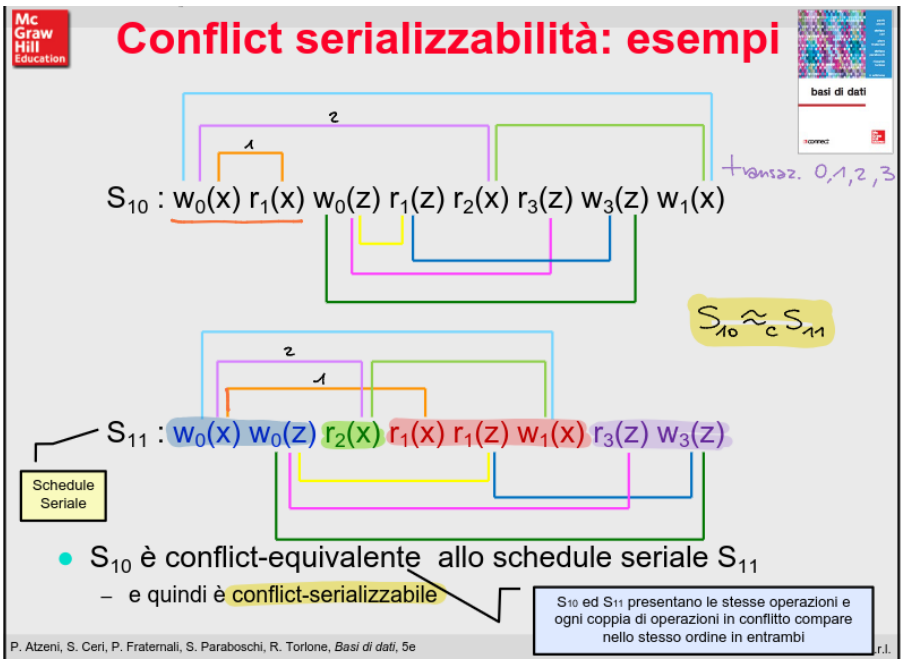


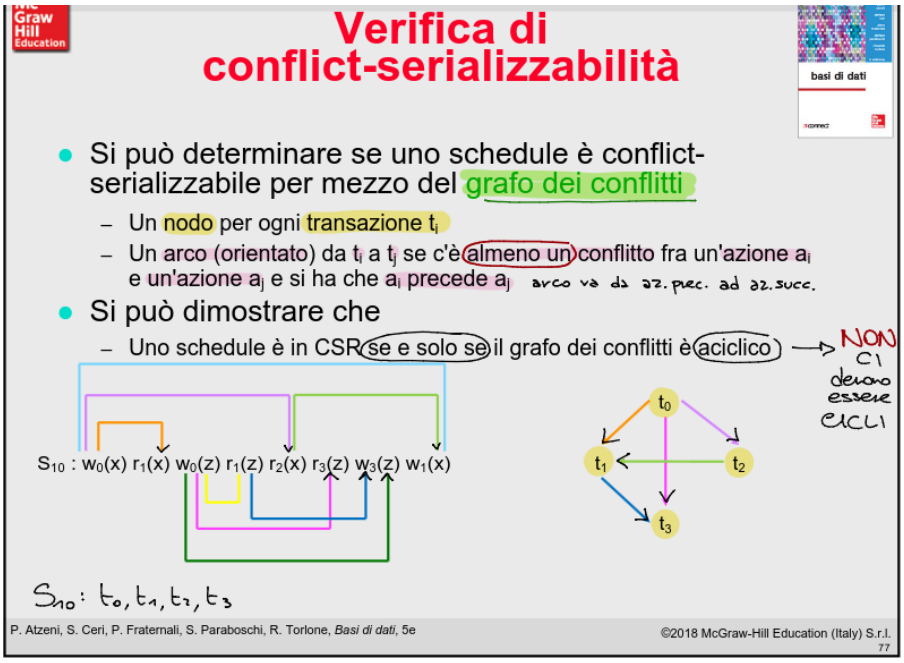
Conflict Equivalenza Nozione di equivalenza più facilmente utilizzabile, si basa sul concetto di **conflitto**. Definizione preliminare:

- Un'azione a_i è in **conflitto** con a_j $(i \neq j)$, se operano sullo stesso oggetto e almeno una di esse è in scrittura. Due casi: conflitto **read-write** (rw o wr) o conflitto **write-write** (ww)

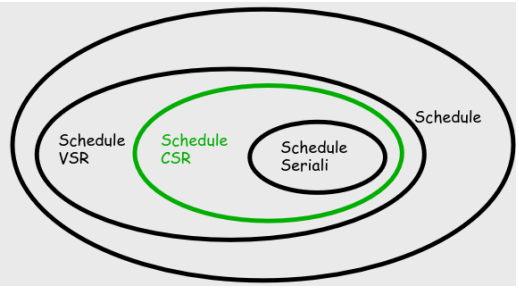
Due schedule sono **conflict equivalenti** $S_i \approx_c S_j$ se presentano le **stesse operazioni** e **ogni coppia di operazioni in conflitto compare nello stesso ordine in entrambi.**

Conflict serializzabilità Uno schedule è **conflict-serializzabile** se è conflict-equivalente ad un qualche schedule seriale. L'insieme degli schedule conflict-serializzabili è indicato con **CSR**.





La classe degli schedule CSR (conflict-serializzabili) è strettamente inclusa in quella degli schedule VSR (view-serializzabili)



Anche la conflict-serializzabilità, pur più rapidamente verificabile è inutilizzabile in pratica: l'algoritmo, con opportune strutture dati richiede tempo lineare, però la conflict-serializzabilità risulta ancora eccessivamente onerosa in pratica. La tecnica sarebbe efficiente se potessimo conoscere il grafo dall'inizio, ma così non è: non è praticabile mantenere il grafo, aggiornarlo e verificarne l'aciclicità ad ogni richiesta di operazione.

5.2.2 Soluzioni utilizzate in pratica: LOCKING

Il meccanismo più noto e il primo usato dai DBMS si basa sul **LOCKING**. In questa soluzione tutte le operazioni di lettura e scrittura devono essere protette da opportune primitive:

- Tutte le letture sono precedute da **r_lock** (lock condiviso) e seguite da **unlock**. Su un dato possono essere attivi più lock di questo tipo.
- Tutte le scritture sono precedute da **w_lock** (lock esclusivo) e seguite da **unlock**. Non può coesistere con altri lock sullo stesso dato.

Quando una transazione segue queste regole si dice **ben formata rispetto ai lock**.
Quando una trasnazione prima legge e poi scrive un oggetto, può richiedere subito un lock esclusivo o chiedere prima un lock condiviso e poi uno esclusivo (lock escalation). Il gestore della concorrenza riceve le richieste di lock provenienti dalle transazioni e le gestisce sulla base della **tavola dei conflitti.**
Se la risorsa non è concessa, la transazione richiedente è posta in attesa (eventualmente in coda), fino a quando la risorsa non diventa disponibile. Il **lock manager** gestisce una tabella dei lock, per ricordare la situazione dei lock già concessi.

5.2.3 Meccanismi di locking

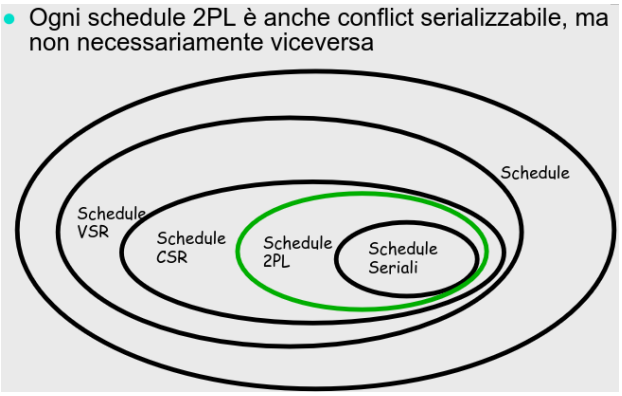
I meccanismi di lockig possono essere usati per garantire le transazioni che lavorano sulla BD accedano ai dati in **mutua esclusione**. Per avere la garanzia che le transazioni seguano uno schedule serializzabile è necessario porre una restrizione sull'ordinamento delle richieste di lock.

Principio di Locking a 2 Fasi - 2PL Usato da quasi tutti i sistemi, garantisce "a priori" la conflict-serializzabilità. È basata su due regole:

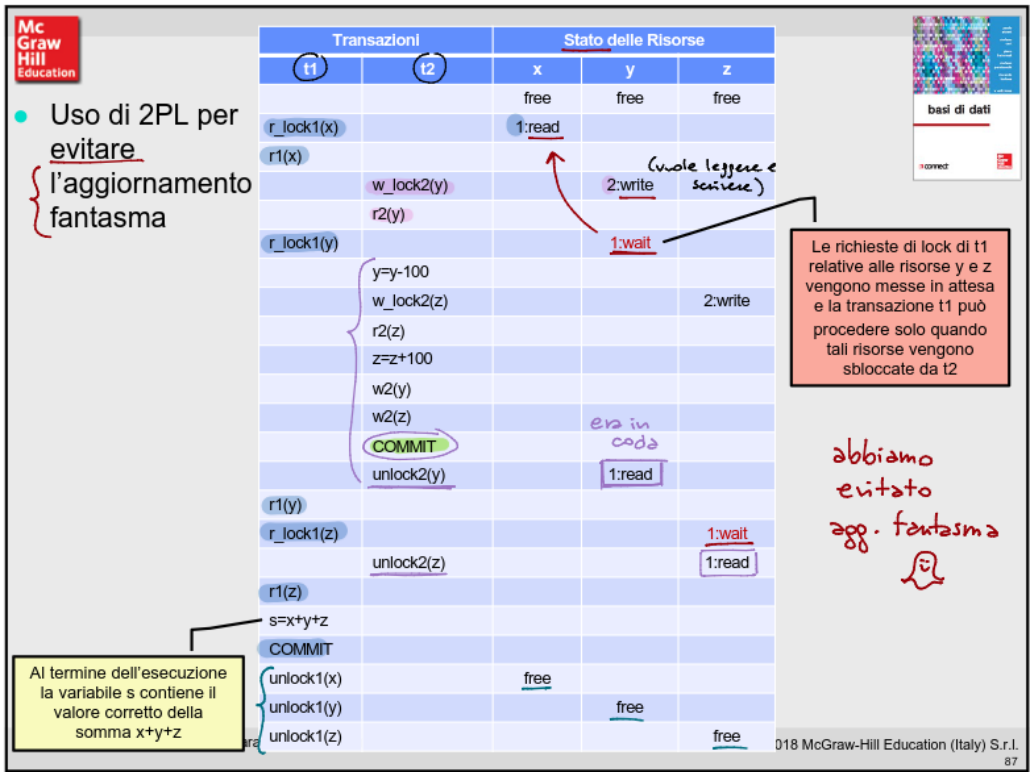
- "proteggere" tutte le letture e scritture con **lock**
- un **vincolo** sulle richieste e i rilasci del lock: una transazione, dopo aver rilasciato un lock, non può acquisirne altri.

Si possono distinguere due fasi nell'esecuzione della transazione:

1. Una prima fase in cui si **acquisiscono i lock** per le risorse a cui si deve accedere - **FASE CRESCENTE** - Passaggio da r_lock a w_lock sta nella fase crescente.
2. Una seconda fase in cui si rilasciano i lock acquisiti - **FASE CALANTE**.



Lo schedule 2Pl è un sistema in cui le **transazioni sono ben formate** rispetto ai locking, cioè richiedono il lock prima di accedere e lo rilasciano prima di terminare con un **Lock Manager** che rispetta la politica nella tabella e in cui le transazioni seguono il principio del locking a due fasi. È un sistema transazionale caratterizzato dalla serializzabilità delle proprie transazioni. La classe 2PL contiene gli schedule che soddisfano queste condizioni.



5.2.4 Locking a due fasi stretto

Restrizione del protocollo 2PL, la condizione aggiuntiva è che **i lock possono essere rilasciati solo dopo il commit o abort**. Permette di rimuovere l'ipotesi *commit-proiezione* ed elimina il rischio di letture sporche. I lock vengono rilasciati solo al termine della transazione, dopo che i dati sono stati portati nel loro stato finale. Versione di 2PL molto usaa nei DBMS commerciali.

5.2.5 Stallo (deadlock) - blocco critico

Avviene quando si hanno delle **attese incrociate**, ovvero quando due transazioni detengono ciascuna una risorsa e aspettano la risorsa detenuta dall'altra. Esempio:

- t_1: read(x), write(y)
- t_2: read(y), write(x)
- Schedule (2PL): $r_lock_1(x), r_lock_2(y), read_1(x), read_2(y) w_lock_1(y), w_lock_2(x)$

Nessuna delle due transazioni riesce a procedure e il sistema è bloccato. t_1 è in attesa che si liberi y che è bloccato da t_2, mentre t_2 è in attesa che si liberi x che è bloccato da t_1. Ci sono tre soluzioni:

1. **Timeout:** tecnica molto semplcie usata in molti DBMS commerciali. Le transazioni rimangono in attesa per un tempo pregissato ragionevole, se passa questo tempo e la risorsa non è stata concess allora alla richiesta di lock viene data risposta negativa. Quindi la transazione in deadlock viene tolta dalla condizione di attesa e presumibilmente abortita.

2. **Rilevamento dello stallo:** questa tecnica prevede di *non porre vincoli* al comportamento del sistema, ma di controllare il contenuto delle tabelle di lock tutte le volte che si ritiene necessario per rilevare eventuali situazioni di blocco. È oneroso e complicato; il rilevamento di un blocco critico richiede di analizzare le relazioni di attesa tra le varie transazioni e di determinare se esiste un ciclo.

3. **Prevenzione dello stallo:** vi sono diverse tecniche per prevenire il blocco critico:

   (a) Richiedere il **lock di tutte le risorse** necessarie alla transazione in una sola volta. Problema: spesso le transazioni non conoscono a priori le risorse a cui vogliono accedere.

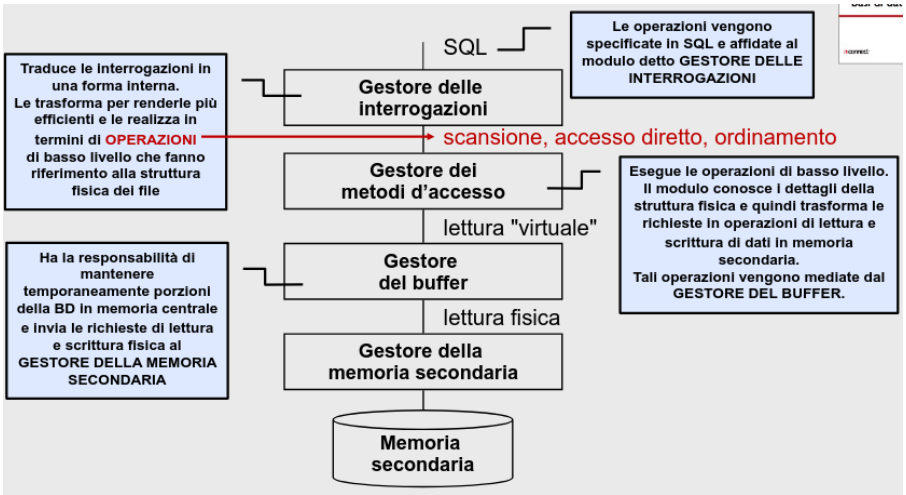
   (b) Le transazioni acquisiscono un timestamp e viene consentita l'attesa di t_i su una risorsa acquisita da t_j solo se vale una certa relazione di precedenza tra i timestamp di t_i e t_j (esempio $i < j$)

(c) Si basa sull'osservazione che molti deadlock si verificano all'atto di "incrementare" un lock da lettura a scrittura: Due transazioni acquisiscono lock condivisi sullo stesso dato e poi cercano di incrementare il lock bloccandosi a vicenda. Soluzione: Introduzione di un nuovo tipo di lock detto di **UPDATE** richiesto per leggere un dato su cui successivamente si vuole scrivere.

UPDATE LOCK: Il deadlock più frequente avviene quando due transazioni concorrenti vogliono prima leggere e poi scrivere la stessa risorsa. Per evitare questa situazione, i sistemi offrono gli update lock (UL). L'update lock viene acquisito da transazioni che vogliono inizialmente leggere un oggetto per poi modificarne il valore. Il lock di update è incompatibile con altri lock di update. In un certo istante una sola transazione può avere un lock di update su un dato. Il lock di update è compatibile con un lock di lettura. All'atto di passare alla scrittura la transazione deve comunque richiedere un lock in scrittura e quindi resta in attesa che eventuali altri lettori terminino la loro azione.

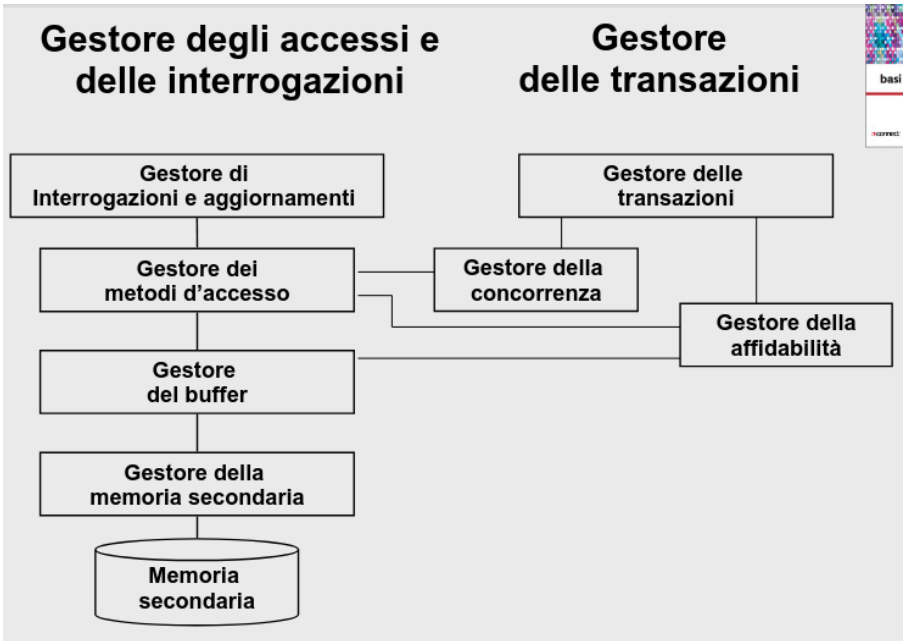
6 Possibili domande d'esame

- ***Si descrivano i moduli che compongono il DBSM descrivendo brevemente le funzioni di ciascuno:*** risposta nelllo schema sottostante.



Bisogna parlare di ciascun modulo brevemente, facendo l'esempio di una query dall'alto verso il basso e viceversa.

- ***(da approfondire) Quali sono i moduli che mi permettono di gestire le transazioni in un DBMS?*** Significa gestire la concorrenza e gestire l'affidabilità.



- ***Si descriva in che modo le tuple vengono organizzate nelle pagine:***

B + Tree F Chiavi, ogni chiave è seguita da un puntatore. La chiave K_1 ha un puntatore prima di sè P_0, quindi abbiamo $F+1$ puntatori che si chiama **Fan Out dell'albero**. (Dato Fan Out =4, abbiamo 3 chiavi). I puntatori saranno sempre uno in più, il Fan Out dice il numero massimo di puntatori che posso avere. In questo caso il puntatore P_0 che sta prima della prima chiave punta al sottoalbero che contiene le chiavi $K < K_1$. P_F punta al sottoalbero che contiene le chiavi $K \geq K_F$ mentre P_i è il puntatore che punta al sottoalbero che contiene le chiavi $K_i \leq K < K_{i+1}$. Data la chiave di ricerca V, ad ogni nodo intermedio, se $V < K$ allora si segue il puntatore P_0, se $V \geq K_F$ si segue il puntatore P_F, altrimenti si segue il puntatore P_j tale che $K_j \leq K < K_{j+1}$. Se l'indice è **primario** i nodi foglia contengono l'intera tupla, se l'indice è secondario i nodi foglia contengono i putnatori ai blocchi della BD che contengono le tuple.
Inserimenti e cancellazioni: nel caso di inserimento o eliminazione di una tupla devo risistemare l'albero affinché rispetti i vincoli. Se c'è posto nella foglia lo inserisco. Se arrivo in un nodo con numero di chiavi

massimo devo dividere il nodo, splittarlo, in modo da inserirlo, ovviamente aggiungendo un nuovo puntatore. Eliminazione, c'è la possibilità di fare un merge. Se devo modificare il valore di una chiave, devo trattarle come eliminazioni seguite da inserimenti.